Relatório técnico acadêmico — Sprint 2 V3: Equilíbrio de carga entre fiscais

Identificação acadêmica

Aluno: Jackson David Rodrigues Penaforte
Curso: Curso Superior de Tecnologia em Logística
Instituição: Faculdade de Tecnologia de São Paulo — FATEC
Unidade: FATEC Jessen Vidal
Cidade: São José dos Campos — SP
Projeto: LOGTEAM 3 / IPEM-SP

1. Objetivo do programa

O programa sprint2_v3_equilibrio_fiscais.py foi desenvolvido para propor uma distribuição mais equilibrada da carga de trabalho entre fiscais. A solução utiliza a base geocodificada produzida na etapa anterior, transforma o histórico de fiscalizações em demanda operacional e monta um plano de distribuição considerando visitas, estabelecimentos, itens fiscalizados, horas de trabalho, quilômetros, municípios, dias úteis e períodos de férias.

2. Participação do aluno no desenvolvimento

A atuação de Jackson David Rodrigues Penaforte está relacionada principalmente à preparação, extração, tratamento e limpeza dos dados que alimentam a solução. Essa participação é fundamental porque a otimização depende de dados consistentes de datas, fiscais, municípios, estabelecimentos, itens, coordenadas, distâncias e tempos. O aluno também auxilia a criação do dashboard, no qual os resultados do plano, do histórico e dos indicadores de equilíbrio podem ser apresentados à equipe e aos usuários. De forma indireta, contribui para a integração entre a base tratada, a lógica de otimização e as visualizações finais.

3. Entradas e saídas

- **Entrada principal:** Base_Geocodificada.xlsx, gerada pelo programa anterior.
- **Entrada opcional:** Ferias_Fiscais_Motoristas.xlsx; se não existir, o programa cria um modelo e considera todos disponíveis.
- **Saída principal:** Sprint_2_V3_Equilibrio.xlsx, com várias abas analíticas.
- **Saída tabular auxiliar:** uma pasta com um CSV por aba.
- **Saídas visuais:** gráficos PNG mensais e por fiscal.

4. Metodologia do processamento

1 Validação e leitura da base geocodificada.
2 Reatribuição de fiscais removidos, preservando a demanda.
3 Construção de calendário, feriados e disponibilidade.
4 Consolidação de visitas e calibração do tempo de fiscalização por faixa de itens.
5 Cálculo da demanda municipal em “saídas”, entendidas como dias de trabalho de uma equipe em um município.
6 Agrupamento dos municípios em zonas geográficas.
7 Otimização MILP para distribuir as saídas entre fiscais.
8 Distribuição espacial dos estabelecimentos dentro de cada município.
9 Criação do plano anual e das metas mensais e semanais.
10 Comparação entre situação atual e plano, geração de indicadores, gráficos e arquivos finais.

5. Critérios de equilíbrio

O modelo procura equilibrar quatro dimensões: **locais**, **itens**, **horas totais** e **quilômetros**. Os pesos configurados são 1,0 para locais, 1,0 para itens, 1,0 para horas e 0,5 para quilômetros. A demanda é atendida integralmente: o objetivo não é reduzir o trabalho existente, mas redistribuí-lo de maneira mais uniforme e proporcional à disponibilidade de cada fiscal.

6. Modelo de otimização

A função alocar_milp usa programação linear inteira mista por meio de scipy.optimize.milp, com o método HiGHS. As variáveis principais representam saídas atribuídas a cada combinação fiscal-município; variáveis auxiliares representam municípios atendidos, zonas utilizadas e desvios máximos. As restrições garantem que toda a demanda seja alocada, limitam a quantidade de fiscais por município, controlam o número de municípios por fiscal, respeitam disponibilidade e evitam blocos pequenos de saídas. Se o MILP falhar, o programa usa uma alocação gulosa como reserva.

7. Tratamento de férias e disponibilidade

O calendário identifica dias úteis, feriados nacionais/estaduais, Carnaval e Corpus Christi. A planilha de férias pode conter vários períodos para a mesma pessoa. Cada intervalo válido transforma a pessoa em indisponível nos dias correspondentes. As metas anuais são depois distribuídas para meses e semanas proporcionalmente aos dias disponíveis.

8. Indicadores produzidos

O programa calcula coeficiente de variação (CV), índice de Gini, médias por dia, percentual de dias acima da jornada máxima, meses sem atividade e comparações entre atual e plano.

Também produz indicadores para horas, quilômetros, itens e locais em níveis diário, semanal, mensal e anual. Valores menores de CV e Gini indicam distribuição mais equilibrada.

9. Abas e arquivos gerados

As principais abas são: Resumo, Parametros, Indicadores_Equilibrio, Plano_Fiscal_Anual, Responsavel_por_Municipio, Matriz_Saidas_Mun_x_Fiscal, Plano_Municipio_Fiscal, Locais_por_Fiscal, Itens_Especie_x_Fiscal, Metas_Fiscal_Mensal, Metas_Fiscal_Semanal, Atual_Mensal_x_Plano, Capacidade_Equipes, Equilibrio_Atual_Fiscais, Demanda_Municipio, Zonas, Tempo_Fiscalizacao, Tempo_por_Itens, Disponibilidade, Ferias_Consideradas e Calendario.

10. Pontos de atenção

- A ausência da planilha de férias não interrompe a execução, mas gera um plano que considera todos disponíveis.
- Fiscais removidos não fazem a demanda desaparecer: seus dias/equipes são reatribuídos a outros fiscais.
- O tempo de fiscalização é inferido dos intervalos entre visitas; registros com geocodificação baixa ou intervalos excessivos são excluídos da calibração.
- Municípios com baixa precisão geográfica ou sem amostra real de viagem recebem estimativas baseadas em distância, circuidade e velocidade.
- O plano representa metas agregadas; a execução diária real ainda depende da programação operacional.
- O limite de 90 segundos para o MILP pode resultar em solução interrompida; por isso existe fallback guloso.

11. Relatório linha por linha

A tabela a seguir lista todas as linhas do programa e explica a finalidade de cada uma. Linhas em branco, comentários e fechamentos sintáticos também foram preservados para manter a correspondência com o arquivo original.

Linha	Código	Explicação
1	"""	Abre a documentação geral do módulo.

Linha	Código	Explicação
2	Sprint 2 - V3 | EQUILÍBRIO DE CARGA entre fiscais | LOGTEAM 3 | IPEM-SP	Identifica a Sprint 2 V3 e o objetivo de equilíbrio de carga entre fiscais no LOGTEAM 3 / IPEM-SP.
3	``	Documenta entradas, saídas e arquivos auxiliares do processamento.
4	Entrada : Base_Geocodificada.xlsx (gerada por gerar_base_geocodificada.py)	Documenta entradas, saídas e arquivos auxiliares do processamento.
5	Ferias_Fiscais_Motoristas.xlsx (opcional; se não existir, o programa cria um MODELO e, por ora,	Documenta entradas, saídas e arquivos auxiliares do processamento.
6	considera que ninguém está de férias)	Documenta entradas, saídas e arquivos auxiliares do processamento.
7	Saídas : Sprint_2_V3_Equilibrio.xlsx (todas as abas)	Documenta entradas, saídas e arquivos auxiliares do processamento.
8	Sprint_2_V3_Equilibrio_csv/ (um .csv por aba, o mesmo conteúdo do .xlsx)	Documenta entradas, saídas e arquivos auxiliares do processamento.
9	Sprint_2_V3_Equilibrio_Mensal.png / _Fiscais.png	Documenta entradas, saídas e arquivos auxiliares do processamento.
10	``	Documenta entradas, saídas e arquivos auxiliares do processamento.
11	O QUE É EQUILIBRADO (para cada fiscal, nos níveis diário, semanal, mensal e anual)	Define as dimensões equilibradas: horas, quilômetros, itens, locais e divisão territorial; compara atual e plano.

Linha	Código	Explicação
12	* horas trabalhadas = tempo nas visitas (média do período, por faixa de itens) + deslocamento	Define as dimensões equilibradas: horas, quilômetros, itens, locais e divisão territorial; compara atual e plano.
13	* distâncias percorridas (km) * itens fiscalizados * locais fiscalizados (visitas)	Define as dimensões equilibradas: horas, quilômetros, itens, locais e divisão territorial; compara atual e plano.
14	e a divisão dos MUNICÍPIOS, dos LOCAIS (estabelecimentos) e dos ITENS entre os fiscais.	Define as dimensões equilibradas: horas, quilômetros, itens, locais e divisão territorial; compara atual e plano.
15	``	Define as dimensões equilibradas: horas, quilômetros, itens, locais e divisão territorial; compara atual e plano.
16	COMO O PLANO É MONTADO	Descreve a metodologia: demanda convertida em saídas, otimização inteira, divisão contígua de estabelecimentos, metas por disponibilidade e indicadores nos níveis diário, semanal, mensal e anual.

Linha	Código	Explicação
17	1) Demanda = o que foi fiscalizado no período, por município, convertido em "saídas" (um dia de trabalho	Descreve a metodologia: demanda convertida em saídas, otimização inteira, divisão contígua de estabelecimentos, metas por disponibilidade e indicadores nos níveis diário, semanal, mensal e anual.
18	de um fiscal em um município: ida da central, visitas, volta).	Descreve a metodologia: demanda convertida em saídas, otimização inteira, divisão contígua de estabelecimentos, metas por disponibilidade e indicadores nos níveis diário, semanal, mensal e anual.
19	2) Otimização (programação linear inteira): distribui as saídas de cada município entre os fiscais,	Descreve a metodologia: demanda convertida em saídas, otimização inteira, divisão contígua de estabelecimentos, metas por disponibilidade e indicadores nos níveis diário, semanal, mensal e anual.
20	atendendo 100% da demanda e minimizando o maior desvio entre fiscais em locais, itens, horas e km,	Descreve a metodologia: demanda convertida em saídas, otimização inteira, divisão contígua de estabelecimentos, metas por disponibilidade e indicadores nos níveis diário, semanal, mensal e anual.

Linha	Código	Explicação
21	proporcional aos dias disponíveis (férias e feriados). Um município pode ter mais de um responsável.	Descreve a metodologia: demanda convertida em saídas, otimização inteira, divisão contígua de estabelecimentos, metas por disponibilidade e indicadores nos níveis diário, semanal, mensal e anual.
22	3) Os estabelecimentos de cada município são repartidos entre os fiscais responsáveis em faixas	Descreve a metodologia: demanda convertida em saídas, otimização inteira, divisão contígua de estabelecimentos, metas por disponibilidade e indicadores nos níveis diário, semanal, mensal e anual.
23	geográficas contíguas, equilibrando visitas, itens e horas.	Descreve a metodologia: demanda convertida em saídas, otimização inteira, divisão contígua de estabelecimentos, metas por disponibilidade e indicadores nos níveis diário, semanal, mensal e anual.
24	4) Metas anuais -> mensais -> semanais, proporcionais aos dias disponíveis de cada fiscal.	Descreve a metodologia: demanda convertida em saídas, otimização inteira, divisão contígua de estabelecimentos, metas por disponibilidade e indicadores nos níveis diário, semanal, mensal e anual.

Linha	Código	Explicação
25	5) Indicadores: equilíbrio ATUAL (período) x PLANO, nos quatro níveis.	Descreve a metodologia: demanda convertida em saídas, otimização inteira, divisão contígua de estabelecimentos, metas por disponibilidade e indicadores nos níveis diário, semanal, mensal e anual.
26	"""	Descreve a metodologia: demanda convertida em saídas, otimização inteira, divisão contígua de estabelecimentos, metas por disponibilidade e indicadores nos níveis diário, semanal, mensal e anual.
27	import os	Fecha a documentação e inicia/importa bibliotecas de sistema, datas, arrays, tabelas, agrupamento hierárquico, otimização MILP e matrizes esparsas.
28	import warnings	Fecha a documentação e inicia/importa bibliotecas de sistema, datas, arrays, tabelas, agrupamento hierárquico, otimização MILP e matrizes esparsas.

Linha	Código	Explicação
29	from collections import OrderedDict	Fecha a documentação e inicia/importa bibliotecas de sistema, datas, arrays, tabelas, agrupamento hierárquico, otimização MILP e matrizes esparsas.
30	from datetime import date, timedelta	Fecha a documentação e inicia/importa bibliotecas de sistema, datas, arrays, tabelas, agrupamento hierárquico, otimização MILP e matrizes esparsas.
31	``	Fecha a documentação e inicia/importa bibliotecas de sistema, datas, arrays, tabelas, agrupamento hierárquico, otimização MILP e matrizes esparsas.
32	import numpy as np	Continuação da operação ou da estrutura de dados do bloco atual.
33	import pandas as pd	Continuação da operação ou da estrutura de dados do bloco atual.
34	from scipy.cluster.hierarchy import fcluster, linkage	Continuação da operação ou da estrutura de dados do bloco atual.
35	from scipy.optimize import Bounds, LinearConstraint, milp	Continuação da operação ou da estrutura de dados do bloco atual.

Linha	Código	Explicação
36	from scipy.sparse import coo_matrix	Continuação da operação ou da estrutura de dados do bloco atual.
37	``	Linha em branco usada para separar blocos.
38	warnings.filterwarnings("ignore", category=FutureWarning)	Suprime alertas FutureWarning para reduzir ruído na execução.
39	``	Linha em branco usada para separar blocos.
40	# ----------------------------- CONFIGURAÇÃO (editável) -----------------------------	Inicia a configuração editável.
41	PASTA = "/content/drive/MyDrive/LOG3/API3/LOGTEAM 3"	Define pasta, arquivos de entrada/saída e exportação CSV por aba.
42	ARQ_BASE = os.path.join(PASTA, "Base_Geocodificada.xlsx")	Define pasta, arquivos de entrada/saída e exportação CSV por aba.
43	ARQ_FERIAS = os.path.join(PASTA, "Ferias_Fiscais_Motoristas.xlsx")	Define pasta, arquivos de entrada/saída e exportação CSV por aba.
44	NOME_SAIDA = "Sprint_2_V3_Equilibrio"	Define pasta, arquivos de entrada/saída e exportação CSV por aba.
45	EXPORTAR_CSV_POR_ABA = True	Define pasta, arquivos de entrada/saída e exportação CSV por aba.

Linha	Código	Explicação
46	``	Linha em branco usada para separar blocos.
47	ANO_PLANO = None # None = ano do histórico	Define ano, fiscais que participarão do plano, fiscais excluídos/removidos e regras de reatribuição.
48	# Fiscais que vão atender TODA a demanda (o plano). Os demais fiscais da base continuam aparecendo no "Atual"	Define ano, fiscais que participarão do plano, fiscais excluídos/removidos e regras de reatribuição.
49	# (como era em 2018), mas o trabalho que fizeram é redistribuído entre os fiscais abaixo.	Define ano, fiscais que participarão do plano, fiscais excluídos/removidos e regras de reatribuição.
50	# Use None para incluir todos os fiscais da base. Os nomes podem estar com ou sem acento e em qualquer caixa.	Define ano, fiscais que participarão do plano, fiscais excluídos/removidos e regras de reatribuição.
51	FISCAIS_PLANO = [	Define ano, fiscais que participarão do plano, fiscais excluídos/removidos e regras de reatribuição.
52	"JOSE LUIZ FERREIRA PINTO",	Define ano, fiscais que participarão do plano, fiscais excluídos/removidos e regras de reatribuição.
53	"ALEXANDRE ABREU FONSECA",	Define ano, fiscais que participarão do plano, fiscais excluídos/removidos e regras de reatribuição.

Linha	Código	Explicação
54	"MARCELO DE ALMEIDA E SILVA",	Define ano, fiscais que participarão do plano, fiscais excluídos/removidos e regras de reatribuição.
55	"PAULO SERGIO BISPO DE AZEVEDO",	Define ano, fiscais que participarão do plano, fiscais excluídos/removidos e regras de reatribuição.
56	"JOSE ETORE DE CONTI",	Define ano, fiscais que participarão do plano, fiscais excluídos/removidos e regras de reatribuição.
57	"FLAVIO DOS SANTOS PEREIRA DUTRA",	Define ano, fiscais que participarão do plano, fiscais excluídos/removidos e regras de reatribuição.
58	"MARCO ANTONIO MARQUES DE SA",	Define ano, fiscais que participarão do plano, fiscais excluídos/removidos e regras de reatribuição.
59	]	Define ano, fiscais que participarão do plano, fiscais excluídos/removidos e regras de reatribuição.
60	FISCAIS_EXCLUIR = [] # tira fiscais do plano mesmo que estejam na lista acima (ex.: licença longa)	Define ano, fiscais que participarão do plano, fiscais excluídos/removidos e regras de reatribuição.
61	# Fiscais que SAEM da planilha (todas as abas, inclusive o "Atual"). O trabalho deles NÃO some: cada dia de equipe	Define ano, fiscais que participarão do plano, fiscais excluídos/removidos e regras de reatribuição.

Linha	Código	Explicação
62	# deles é reatribuído a outro fiscal da base (que não trabalhou naquele dia e que mais atua no município). Assim o total	Define ano, fiscais que participarão do plano, fiscais excluídos/removidos e regras de reatribuição.
63	# de visitas/itens e a demanda do plano ficam idênticos; só muda quem aparece como executor no "Atual".	Define ano, fiscais que participarão do plano, fiscais excluídos/removidos e regras de reatribuição.
64	FISCAIS_REMOVER = [	Define ano, fiscais que participarão do plano, fiscais excluídos/removidos e regras de reatribuição.
65	"DANIEL ESTEVES SANTANA",	Define ano, fiscais que participarão do plano, fiscais excluídos/removidos e regras de reatribuição.
66	"CARLA CRISTINA LOPES PEREIRA DE MORAES",	Define ano, fiscais que participarão do plano, fiscais excluídos/removidos e regras de reatribuição.
67	"GILSON MATEUS DAMAS",	Define ano, fiscais que participarão do plano, fiscais excluídos/removidos e regras de reatribuição.
68	"ADEMIR GONCALVES PEREIRA",	Define ano, fiscais que participarão do plano, fiscais excluídos/removidos e regras de reatribuição.
69	"MARCIA CRISTINA LORENZ SANTOS",	Define ano, fiscais que participarão do plano, fiscais excluídos/removidos e regras de reatribuição.

Linha	Código	Explicação
70	"LUIS ANTONIO GODINHO DA SILVA",	Define ano, fiscais que participarão do plano, fiscais excluídos/removidos e regras de reatribuição.
71	"RENATO FERREIRA DE LIMA",	Define ano, fiscais que participarão do plano, fiscais excluídos/removidos e regras de reatribuição.
72	]	Define ano, fiscais que participarão do plano, fiscais excluídos/removidos e regras de reatribuição.
73	``	Linha em branco usada para separar blocos.
74	# central da regional (início e fim do dia) - usada para estimar retorno/viagem quando a base não traz	Define coordenadas/endereço da sede, fator de circuidade e velocidade média de fallback.
75	SEDE = (-23.180604, -45.882922)	Define coordenadas/endereço da sede, fator de circuidade e velocidade média de fallback.
76	SEDE_ENDERECO = "Rua Sebastião Hummel, 759 - Centro, São José dos Campos - SP (CEP 12210-200)"	Define coordenadas/endereço da sede, fator de circuidade e velocidade média de fallback.
77	FATOR_CIRCUIDADE = 1.30 # linha reta -> estrada (só nas estimativas)	Define coordenadas/endereço da sede, fator de circuidade e velocidade média de fallback.

Linha	Código	Explicação
78	VELOCIDADE_KMH = 40.0	Define coordenadas/endereço da sede, fator de circuidade e velocidade média de fallback.
79	``	Linha em branco usada para separar blocos.
80	# tempo de fiscalização por visita (calibrado no período)	Define calibração do tempo de fiscalização, jornada máxima, regras de viagem, ajuste de compartilhamento e normalização de municípios.
81	TEMPO_FISCALIZACAO_MANUAL_MIN = None # ex.: 25 força o mesmo tempo para todas as faixas	Define calibração do tempo de fiscalização, jornada máxima, regras de viagem, ajuste de compartilhamento e normalização de municípios.
82	LIMITE_INTERVALO_MIN = 180	Define calibração do tempo de fiscalização, jornada máxima, regras de viagem, ajuste de compartilhamento e normalização de municípios.
83	MIN_PARES_CALIBRACAO = 200	Define calibração do tempo de fiscalização, jornada máxima, regras de viagem, ajuste de compartilhamento e normalização de municípios.

Linha	Código	Explicação
84	MIN_PARES_FAIXA = 30	Define calibração do tempo de fiscalização, jornada máxima, regras de viagem, ajuste de compartilhamento e normalização de municípios.
85	``	Linha em branco usada para separar blocos.
86	JORNADA_MAX_H = 8.0 # limite de horas de uma saída	Define calibração do tempo de fiscalização, jornada máxima, regras de viagem, ajuste de compartilhamento e normalização de municípios.
87	MIN_AMOSTRA_VIAGEM = 1 # nº mínimo de trechos reais (1º do dia / retorno) para usar a rota da base num município	Define calibração do tempo de fiscalização, jornada máxima, regras de viagem, ajuste de compartilhamento e normalização de municípios.
88	AJUSTAR_COMPARTILHAMENTO = True # dias que juntaram mais de um município dividem a viagem à central (como no período)	Define calibração do tempo de fiscalização, jornada máxima, regras de viagem, ajuste de compartilhamento e normalização de municípios.
89	FATOR_DEMANDA = 1.0 # 1.0 = mesma demanda do período	Define calibração do tempo de fiscalização, jornada máxima, regras de viagem, ajuste de compartilhamento e normalização de municípios.

Linha	Código	Explicação
90	GEO_BAIXA = {"MUNICIPIO", "BAIRRO", "NAO_ENCONTRADO"}	Define calibração do tempo de fiscalização, jornada máxima, regras de viagem, ajuste de compartilhamento e normalização de municípios.
91	AJUSTES_NOME_MUN = {"MIRIM": "BIRITIBA-MIRIM"}	Define calibração do tempo de fiscalização, jornada máxima, regras de viagem, ajuste de compartilhamento e normalização de municípios.
92	``	Linha em branco usada para separar blocos.
93	# divisão dos municípios	Define parâmetros de zonas, limites de fiscais/municípios, pesos da otimização, tolerâncias, tempo-limite e feriados extras.
94	N_ZONAS = 5	Define parâmetros de zonas, limites de fiscais/municípios, pesos da otimização, tolerâncias, tempo-limite e feriados extras.
95	MAX_FISCAIS_POR_MUN = 4	Define parâmetros de zonas, limites de fiscais/municípios, pesos da otimização, tolerâncias, tempo-limite e feriados extras.

Linha	Código	Explicação
96	MIN_MUN_POR_FISCAL = 3	Define parâmetros de zonas, limites de fiscais/municípios, pesos da otimização, tolerâncias, tempo-limite e feriados extras.
97	MAX_MUN_POR_FISCAL = 9	Define parâmetros de zonas, limites de fiscais/municípios, pesos da otimização, tolerâncias, tempo-limite e feriados extras.
98	MIN_SAIDAS_BLOCO = 3	Define parâmetros de zonas, limites de fiscais/municípios, pesos da otimização, tolerâncias, tempo-limite e feriados extras.
99	USO_MAX_DIAS = 1.0	Define parâmetros de zonas, limites de fiscais/municípios, pesos da otimização, tolerâncias, tempo-limite e feriados extras.
100	PESOS = {"LOCAIS": 1.0, "ITENS": 1.0, "H_TOTAL": 1.0, "KM": 0.5}	Define parâmetros de zonas, limites de fiscais/municípios, pesos da otimização, tolerâncias, tempo-limite e feriados extras.
101	EPS_PAR = 1e-4	Define parâmetros de zonas, limites de fiscais/municípios, pesos da otimização, tolerâncias, tempo-limite e feriados extras.

Linha	Código	Explicação
102	EPS_ZONA = 5e-4	Define parâmetros de zonas, limites de fiscais/municípios, pesos da otimização, tolerâncias, tempo-limite e feriados extras.
103	TEMPO_LIMITE_S = 90	Define parâmetros de zonas, limites de fiscais/municípios, pesos da otimização, tolerâncias, tempo-limite e feriados extras.
104	TOLERANCIA_META = 0.10	Define parâmetros de zonas, limites de fiscais/municípios, pesos da otimização, tolerâncias, tempo-limite e feriados extras.
105	``	Linha em branco usada para separar blocos.
106	FERIADOS_FACULTATIVOS = True	Define parâmetros de zonas, limites de fiscais/municípios, pesos da otimização, tolerâncias, tempo-limite e feriados extras.
107	FERIADOS_EXTRAS = []	Define parâmetros de zonas, limites de fiscais/municípios, pesos da otimização, tolerâncias, tempo-limite e feriados extras.
108	``	Linha em branco usada para separar blocos.

Linha	Código	Explicação
109	``	Linha em branco usada para separar blocos.
110	# ---------- UTILIDADES, CALENDÁRIO E FÉRIAS ----------	Inicia utilidades de calendário, nomes e férias.
111	def haversine_km(lat1, lon1, lat2, lon2):	Calcula a distância Haversine em quilômetros de forma vetorizada.
112	"""Distância em linha reta (km), vetorizada; propaga NaN."""	Calcula a distância Haversine em quilômetros de forma vetorizada.
113	r = 6371.0	Calcula a distância Haversine em quilômetros de forma vetorizada.
114	p1, p2 = np.radians(lat1), np.radians(lat2)	Calcula a distância Haversine em quilômetros de forma vetorizada.
115	dp, dl = p2 - p1, np.radians(lon2 - lon1)	Calcula a distância Haversine em quilômetros de forma vetorizada.
116	a = np.sin(dp / 2) ** 2 + np.cos(p1) * np.cos(p2) * np.sin(dl / 2) ** 2	Calcula a distância Haversine em quilômetros de forma vetorizada.
117	return 2 * r * np.arcsin(np.sqrt(a))	Calcula a distância Haversine em quilômetros de forma vetorizada.
118	``	Linha em branco usada para separar blocos.
119	``	Linha em branco usada para separar blocos.

Linha	Código	Explicação
120	def _norm_nome(s: str) -> str:	Normaliza nomes: remove acentos, converte para maiúsculas e reduz espaços duplicados.
121	"""Maiúsculas, sem acento e sem espaços duplos (para comparar nomes)"""	Normaliza nomes: remove acentos, converte para maiúsculas e reduz espaços duplicados.
122	import unicodedata	Normaliza nomes: remove acentos, converte para maiúsculas e reduz espaços duplicados.
123	t = unicodedata.normalize("NFKD", str(s)).encode("ascii", "ignore").decode("ascii")	Normaliza nomes: remove acentos, converte para maiúsculas e reduz espaços duplicados.
124	return " ".join(t.upper().split())	Normaliza nomes: remove acentos, converte para maiúsculas e reduz espaços duplicados.
125	``	Normaliza nomes: remove acentos, converte para maiúsculas e reduz espaços duplicados.
126	``	Linha em branco usada para separar blocos.
127	def resolver_fiscais(lista, existentes):	Resolve nomes informados contra os nomes existentes; falha explicitamente se algum fiscal do plano não for encontrado.

Linha	Código	Explicação
128	"""Casa os nomes pedidos com os nomes da base; avisa os que não forem encontrados."""	Resolve nomes informados contra os nomes existentes; falha explicitamente se algum fiscal do plano não for encontrado.
129	por_norma = {_norm_nome(f): f for f in existentes}	Resolve nomes informados contra os nomes existentes; falha explicitamente se algum fiscal do plano não for encontrado.
130	achados, faltam = [], []	Resolve nomes informados contra os nomes existentes; falha explicitamente se algum fiscal do plano não for encontrado.
131	for nome in lista:	Resolve nomes informados contra os nomes existentes; falha explicitamente se algum fiscal do plano não for encontrado.
132	f = por_norma.get(_norm_nome(nome))	Resolve nomes informados contra os nomes existentes; falha explicitamente se algum fiscal do plano não for encontrado.
133	(achados if f else faltam).append(f or nome)	Resolve nomes informados contra os nomes existentes; falha explicitamente se algum fiscal do plano não for encontrado.

Linha	Código	Explicação
134	`    if faltam:`	Resolve nomes informados contra os nomes existentes; falha explicitamente se algum fiscal do plano não for encontrado.
135	`        raise RuntimeError(f"Fiscais não encontrados na base: {faltam}. Nomes disponíveis: {sorted(existentes)}")`	Resolve nomes informados contra os nomes existentes; falha explicitamente se algum fiscal do plano não for encontrado.
136	`    return sorted(set(achados))`	Resolve nomes informados contra os nomes existentes; falha explicitamente se algum fiscal do plano não for encontrado.
137	``	Linha em branco usada para separar blocos.
138	``	Linha em branco usada para separar blocos.
139	`def reatribuir_fiscais_removidos(df: pd.DataFrame, remover: list) -> pd.DataFrame:`	Remove fiscais selecionados sem perder demanda: registra o fiscal original, identifica equipes-dia, reaproveita parceiro ou escolhe fiscal disponível com maior atuação no município e menor carga, e atualiza o executor.

Linha	Código	Explicação
140	"""Tira os fiscais de \remover` da base sem perder nenhuma fiscalização.`	Remove fiscais selecionados sem perder demanda: registra o fiscal original, identifica equipes-dia, reaproveita parceiro ou escolhe fiscal disponível com maior atuação no município e menor carga, e atualiza o executor.
141	Cada equipe-dia deles passa a outro fiscal: o parceiro que já estava na mesma equipe/dia, se houver; senão o fiscal	Remove fiscais selecionados sem perder demanda: registra o fiscal original, identifica equipes-dia, reaproveita parceiro ou escolhe fiscal disponível com maior atuação no município e menor carga, e atualiza o executor.
142	que NÃO trabalhou naquele dia e que mais atua no município (desempate: menor carga). Guarda o original em FISCAL_ORIG."""	Remove fiscais selecionados sem perder demanda: registra o fiscal original, identifica equipes-dia, reaproveita parceiro ou escolhe fiscal disponível com maior atuação no município e menor carga, e atualiza o executor.
143	df = df.copy()	Remove fiscais selecionados sem perder demanda: registra o fiscal original, identifica equipes-dia, reaproveita parceiro ou escolhe fiscal disponível com maior atuação no município e menor carga, e atualiza o executor.

Linha	Código	Explicação
144	`    df["FISCAL_ORIG"] = df["FISCAL"]`	Remove fiscais selecionados sem perder demanda: registra o fiscal original, identifica equipes-dia, reaproveita parceiro ou escolhe fiscal disponível com maior atuação no município e menor carga, e atualiza o executor.
145	`    existentes = sorted(df["FISCAL"].unique())`	Remove fiscais selecionados sem perder demanda: registra o fiscal original, identifica equipes-dia, reaproveita parceiro ou escolhe fiscal disponível com maior atuação no município e menor carga, e atualiza o executor.
146	`    por_norma = {_norm_nome(f): f for f in existentes}`	Remove fiscais selecionados sem perder demanda: registra o fiscal original, identifica equipes-dia, reaproveita parceiro ou escolhe fiscal disponível com maior atuação no município e menor carga, e atualiza o executor.
147	`    alvo, faltam = set(), []`	Remove fiscais selecionados sem perder demanda: registra o fiscal original, identifica equipes-dia, reaproveita parceiro ou escolhe fiscal disponível com maior atuação no município e menor carga, e atualiza o executor.

Linha	Código	Explicação
148	`    for nome in remover or []:`	Remove fiscais selecionados sem perder demanda: registra o fiscal original, identifica equipes-dia, reaproveita parceiro ou escolhe fiscal disponível com maior atuação no município e menor carga, e atualiza o executor.
149	`        f = por_norma.get(_norm_nome(nome))`	Remove fiscais selecionados sem perder demanda: registra o fiscal original, identifica equipes-dia, reaproveita parceiro ou escolhe fiscal disponível com maior atuação no município e menor carga, e atualiza o executor.
150	`        (alvo.add(f) if f else faltam.append(nome))`	Remove fiscais selecionados sem perder demanda: registra o fiscal original, identifica equipes-dia, reaproveita parceiro ou escolhe fiscal disponível com maior atuação no município e menor carga, e atualiza o executor.
151	`    if faltam:`	Remove fiscais selecionados sem perder demanda: registra o fiscal original, identifica equipes-dia, reaproveita parceiro ou escolhe fiscal disponível com maior atuação no município e menor carga, e atualiza o executor.

Linha	Código	Explicação
152	print("AVISO: fiscais para remover que não estão na base (ignorados):", faltam)	Remove fiscais selecionados sem perder demanda: registra o fiscal original, identifica equipes-dia, reaproveita parceiro ou escolhe fiscal disponível com maior atuação no município e menor carga, e atualiza o executor.
153	if not alvo:	Remove fiscais selecionados sem perder demanda: registra o fiscal original, identifica equipes-dia, reaproveita parceiro ou escolhe fiscal disponível com maior atuação no município e menor carga, e atualiza o executor.
154	return df	Remove fiscais selecionados sem perder demanda: registra o fiscal original, identifica equipes-dia, reaproveita parceiro ou escolhe fiscal disponível com maior atuação no município e menor carga, e atualiza o executor.
155	restantes = [f for f in existentes if f not in alvo]	Remove fiscais selecionados sem perder demanda: registra o fiscal original, identifica equipes-dia, reaproveita parceiro ou escolhe fiscal disponível com maior atuação no município e menor carga, e atualiza o executor.

Linha	Código	Explicação
156	`    if not restantes:`	Remove fiscais selecionados sem perder demanda: registra o fiscal original, identifica equipes-dia, reaproveita parceiro ou escolhe fiscal disponível com maior atuação no município e menor carga, e atualiza o executor.
157	`        raise RuntimeError("FISCAIS_REMOVER removeria todos os fiscais da base.")`	Remove fiscais selecionados sem perder demanda: registra o fiscal original, identifica equipes-dia, reaproveita parceiro ou escolhe fiscal disponível com maior atuação no município e menor carga, e atualiza o executor.
158	`    chave = ["ID_EQUIPE", "DIA"]`	Remove fiscais selecionados sem perder demanda: registra o fiscal original, identifica equipes-dia, reaproveita parceiro ou escolhe fiscal disponível com maior atuação no município e menor carga, e atualiza o executor.
159	`    sai = df["FISCAL"].isin(alvo)`	Remove fiscais selecionados sem perder demanda: registra o fiscal original, identifica equipes-dia, reaproveita parceiro ou escolhe fiscal disponível com maior atuação no município e menor carga, e atualiza o executor.

Linha	Código	Explicação
160	``	Remove fiscais selecionados sem perder demanda: registra o fiscal original, identifica equipes-dia, reaproveita parceiro ou escolhe fiscal disponível com maior atuação no município e menor carga, e atualiza o executor.
161	# 1) equipe-dia que também tinha um fiscal que fica: o trabalho vai para ele	Remove fiscais selecionados sem perder demanda: registra o fiscal original, identifica equipes-dia, reaproveita parceiro ou escolhe fiscal disponível com maior atuação no município e menor carga, e atualiza o executor.
162	parceiro = (df[~sai].groupby(chave)["FISCAL"].agg(lambda s: s.value_counts().idxmax()).rename("DESTINO").reset_index())	Remove fiscais selecionados sem perder demanda: registra o fiscal original, identifica equipes-dia, reaproveita parceiro ou escolhe fiscal disponível com maior atuação no município e menor carga, e atualiza o executor.
163	grupos = df[sai].groupby(chave).agg(MUN=("MUNICIPIO", lambda s: s.value_counts().idxmax()), N=("CODITEM", "size")).reset_index()	Remove fiscais selecionados sem perder demanda: registra o fiscal original, identifica equipes-dia, reaproveita parceiro ou escolhe fiscal disponível com maior atuação no município e menor carga, e atualiza o executor.

Linha	Código	Explicação
164	`    grupos = grupos.merge(parceiro, on=chave, how="left")`	Remove fiscais selecionados sem perder demanda: registra o fiscal original, identifica equipes-dia, reaproveita parceiro ou escolhe fiscal disponível com maior atuação no município e menor carga, e atualiza o executor.
165	```	Remove fiscais selecionados sem perder demanda: registra o fiscal original, identifica equipes-dia, reaproveita parceiro ou escolhe fiscal disponível com maior atuação no município e menor carga, e atualiza o executor.
166	`    # 2) equipe-dia só de removidos: escolhe quem não trabalhou no dia, que mais atua no município e tem menos carga`	Remove fiscais selecionados sem perder demanda: registra o fiscal original, identifica equipes-dia, reaproveita parceiro ou escolhe fiscal disponível com maior atuação no município e menor carga, e atualiza o executor.
167	`    ocupado = set(map(tuple, df.loc[~sai, ["FISCAL", "DIA"]].drop_duplicates().to_numpy()))`	Remove fiscais selecionados sem perder demanda: registra o fiscal original, identifica equipes-dia, reaproveita parceiro ou escolhe fiscal disponível com maior atuação no município e menor carga, e atualiza o executor.

Linha	Código	Explicação
168	`    afin = df[~sai].groupby(["FISCAL", "MUNICIPIO"]).size().to_dict()`	Remove fiscais selecionados sem perder demanda: registra o fiscal original, identifica equipes-dia, reaproveita parceiro ou escolhe fiscal disponível com maior atuação no município e menor carga, e atualiza o executor.
169	`    carga = df[~sai].groupby("FISCAL").size().reindex(restantes).fillna(0).to_dict()`	Remove fiscais selecionados sem perder demanda: registra o fiscal original, identifica equipes-dia, reaproveita parceiro ou escolhe fiscal disponível com maior atuação no município e menor carga, e atualiza o executor.
170	`    for i in grupos.index[grupos["DESTINO"].isna()].tolist():`	Remove fiscais selecionados sem perder demanda: registra o fiscal original, identifica equipes-dia, reaproveita parceiro ou escolhe fiscal disponível com maior atuação no município e menor carga, e atualiza o executor.
171	`        g = grupos.loc[i]`	Remove fiscais selecionados sem perder demanda: registra o fiscal original, identifica equipes-dia, reaproveita parceiro ou escolhe fiscal disponível com maior atuação no município e menor carga, e atualiza o executor.

Linha	Código	Explicação
172	`        d = min(restantes, key=lambda f: ((f, g["DIA"]) in ocupado, -afin.get((f, g["MUN"]), 0), carga[f]))`	Remove fiscais selecionados sem perder demanda: registra o fiscal original, identifica equipes-dia, reaproveita parceiro ou escolhe fiscal disponível com maior atuação no município e menor carga, e atualiza o executor.
173	`        grupos.loc[i, "DESTINO"] = d`	Remove fiscais selecionados sem perder demanda: registra o fiscal original, identifica equipes-dia, reaproveita parceiro ou escolhe fiscal disponível com maior atuação no município e menor carga, e atualiza o executor.
174	`        ocupado.add((d, g["DIA"]))`	Remove fiscais selecionados sem perder demanda: registra o fiscal original, identifica equipes-dia, reaproveita parceiro ou escolhe fiscal disponível com maior atuação no município e menor carga, e atualiza o executor.
175	`        carga[d] += g["N"]`	Remove fiscais selecionados sem perder demanda: registra o fiscal original, identifica equipes-dia, reaproveita parceiro ou escolhe fiscal disponível com maior atuação no município e menor carga, e atualiza o executor.

Linha	Código	Explicação
176	`    mapa = grupos.set_index(chave)["DESTINO"]`	Remove fiscais selecionados sem perder demanda: registra o fiscal original, identifica equipes-dia, reaproveita parceiro ou escolhe fiscal disponível com maior atuação no município e menor carga, e atualiza o executor.
177	`    novo = df.loc[sai, chave].merge(mapa.reset_index(), on=chave, how="left")["DESTINO"].to_numpy()`	Remove fiscais selecionados sem perder demanda: registra o fiscal original, identifica equipes-dia, reaproveita parceiro ou escolhe fiscal disponível com maior atuação no município e menor carga, e atualiza o executor.
178	`    df.loc[sai, "FISCAL"] = novo`	Remove fiscais selecionados sem perder demanda: registra o fiscal original, identifica equipes-dia, reaproveita parceiro ou escolhe fiscal disponível com maior atuação no município e menor carga, e atualiza o executor.
179	`    return df`	Remove fiscais selecionados sem perder demanda: registra o fiscal original, identifica equipes-dia, reaproveita parceiro ou escolhe fiscal disponível com maior atuação no município e menor carga, e atualiza o executor.

Linha	Código	Explicação
180	``	Linha em branco usada para separar blocos.
181	``	Linha em branco usada para separar blocos.
182	def cv(x) -> float:	Calcula o coeficiente de variação, ignorando NaN e retornando NaN quando não há média válida.
183	x = np.asarray(x, dtype=float)	Calcula o coeficiente de variação, ignorando NaN e retornando NaN quando não há média válida.
184	x = x[~np.isnan(x)]	Calcula o coeficiente de variação, ignorando NaN e retornando NaN quando não há média válida.
185	if len(x) == 0 or x.mean() == 0:	Calcula o coeficiente de variação, ignorando NaN e retornando NaN quando não há média válida.
186	return np.nan	Calcula o coeficiente de variação, ignorando NaN e retornando NaN quando não há média válida.
187	return float(x.std(ddof=0) / x.mean())	Calcula o coeficiente de variação, ignorando NaN e retornando NaN quando não há média válida.
188	``	Linha em branco usada para separar blocos.

Linha	Código	Explicação
189	``	Linha em branco usada para separar blocos.
190	def gini(x) -> float:	Calcula o índice de Gini para medir desigualdade da distribuição.
191	x = np.sort(np.asarray(x, dtype=float))	Calcula o índice de Gini para medir desigualdade da distribuição.
192	n = len(x)	Calcula o índice de Gini para medir desigualdade da distribuição.
193	if n == 0 or x.sum() == 0:	Calcula o índice de Gini para medir desigualdade da distribuição.
194	return np.nan	Calcula o índice de Gini para medir desigualdade da distribuição.
195	return float((2 * np.sum(np.arange(1, n + 1) * x)) / (n * x.sum()) - (n + 1) / n)	Calcula o índice de Gini para medir desigualdade da distribuição.
196	``	Linha em branco usada para separar blocos.
197	``	Linha em branco usada para separar blocos.
198	def distribuir_inteiro(total, pesos):	Distribui um total inteiro proporcionalmente aos pesos pelo método dos maiores restos.

Linha	Código	Explicação
199	"""Reparte um total inteiro em partes inteiras proporcionais aos pesos (maiores restos)"""	Distribui um total inteiro proporcionalmente aos pesos pelo método dos maiores restos.
200	pesos = np.asarray(pesos, dtype=float)	Distribui um total inteiro proporcionalmente aos pesos pelo método dos maiores restos.
201	total = int(round(total))	Distribui um total inteiro proporcionalmente aos pesos pelo método dos maiores restos.
202	if pesos.sum() <= 0:	Distribui um total inteiro proporcionalmente aos pesos pelo método dos maiores restos.
203	return np.zeros(len(pesos), dtype=int)	Distribui um total inteiro proporcionalmente aos pesos pelo método dos maiores restos.
204	ideal = total * pesos / pesos.sum()	Distribui um total inteiro proporcionalmente aos pesos pelo método dos maiores restos.
205	base = np.floor(ideal).astype(int)	Distribui um total inteiro proporcionalmente aos pesos pelo método dos maiores restos.
206	resto = total - int(base.sum())	Distribui um total inteiro proporcionalmente aos pesos pelo método dos maiores restos.

Linha	Código	Explicação
207	`    if resto > 0:`	Distribui um total inteiro proporcionalmente aos pesos pelo método dos maiores restos.
208	`        base[np.argsort(-(ideal - base))[:resto]] += 1`	Distribui um total inteiro proporcionalmente aos pesos pelo método dos maiores restos.
209	`    return base`	Distribui um total inteiro proporcionalmente aos pesos pelo método dos maiores restos.
210	``	Linha em branco usada para separar blocos.
211	``	Linha em branco usada para separar blocos.
212	`def arredondar(df: pd.DataFrame, casas: int = 3) -> pd.DataFrame:`	Arredonda colunas de ponto flutuante sem modificar o dataframe de origem.
213	`    df = df.copy()`	Arredonda colunas de ponto flutuante sem modificar o dataframe de origem.
214	`    cols = df.select_dtypes(include="float").columns`	Arredonda colunas de ponto flutuante sem modificar o dataframe de origem.
215	`    df[cols] = df[cols].round(casas)`	Arredonda colunas de ponto flutuante sem modificar o dataframe de origem.

Linha	Código	Explicação
216	`    return df`	Arredonda colunas de ponto flutuante sem modificar o dataframe de origem.
217	``	Linha em branco usada para separar blocos.
218	``	Linha em branco usada para separar blocos.
219	`def pascoa(ano: int) -> date:`	Calcula a data da Páscoa pelo algoritmo de Meeus/Jones/Butcher.
220	`    a, b, c = ano % 19, ano // 100, ano % 100`	Calcula a data da Páscoa pelo algoritmo de Meeus/Jones/Butcher.
221	`    d, e = b // 4, b % 4`	Calcula a data da Páscoa pelo algoritmo de Meeus/Jones/Butcher.
222	`    f = (b + 8) // 25`	Calcula a data da Páscoa pelo algoritmo de Meeus/Jones/Butcher.
223	`    g = (b - f + 1) // 3`	Calcula a data da Páscoa pelo algoritmo de Meeus/Jones/Butcher.
224	`    h = (19 * a + b - d - g + 15) % 30`	Calcula a data da Páscoa pelo algoritmo de Meeus/Jones/Butcher.

Linha	Código	Explicação
225	`    i, k = c // 4, c % 4`	Calcula a data da Páscoa pelo algoritmo de Meeus/Jones/Butcher.
226	`    l = (32 + 2 * e + 2 * i - h - k) % 7`	Calcula a data da Páscoa pelo algoritmo de Meeus/Jones/Butcher.
227	`    m = (a + 11 * h + 22 * l) // 451`	Calcula a data da Páscoa pelo algoritmo de Meeus/Jones/Butcher.
228	`    mes = (h + l - 7 * m + 114) // 31`	Calcula a data da Páscoa pelo algoritmo de Meeus/Jones/Butcher.
229	`    dia = ((h + l - 7 * m + 114) % 31) + 1`	Calcula a data da Páscoa pelo algoritmo de Meeus/Jones/Butcher.
230	`    return date(ano, mes, dia)`	Calcula a data da Páscoa pelo algoritmo de Meeus/Jones/Butcher.
231	``	Linha em branco usada para separar blocos.
232	``	Linha em branco usada para separar blocos.
233	`def feriados(ano: int) -> dict:`	Monta o dicionário de feriados nacionais/estaduais, facultativos e adicionais.

Linha	Código	Explicação
234	p = pascoa(ano)	Monta o dicionário de feriados nacionais/estaduais, facultativos e adicionais.
235	f = {date(ano, 1, 1): "Confraternização Universal", date(ano, 4, 21): "Tiradentes",	Monta o dicionário de feriados nacionais/estaduais, facultativos e adicionais.
236	date(ano, 5, 1): "Dia do Trabalho", date(ano, 7, 9): "Revolução Constitucionalista (SP)",	Monta o dicionário de feriados nacionais/estaduais, facultativos e adicionais.
237	date(ano, 9, 7): "Independência", date(ano, 10, 12): "Nossa Senhora Aparecida",	Monta o dicionário de feriados nacionais/estaduais, facultativos e adicionais.
238	date(ano, 11, 2): "Finados", date(ano, 11, 15): "Proclamação da República",	Monta o dicionário de feriados nacionais/estaduais, facultativos e adicionais.
239	date(ano, 12, 25): "Natal", p - timedelta(days=2): "Sexta-feira Santa"}	Monta o dicionário de feriados nacionais/estaduais, facultativos e adicionais.
240	if FERIADOS_FACULTATIVOS:	Monta o dicionário de feriados nacionais/estaduais, facultativos e adicionais.
241	f[p - timedelta(days=48)] = "Carnaval (segunda)"	Monta o dicionário de feriados nacionais/estaduais, facultativos e adicionais.
242	f[p - timedelta(days=47)] = "Carnaval (terça)"	Monta o dicionário de feriados nacionais/estaduais, facultativos e adicionais.

Linha	Código	Explicação
243	`        f[p + timedelta(days=60)] = "Corpus Christi"`	Monta o dicionário de feriados nacionais/estaduais, facultativos e adicionais.
244	`    for d in FERIADOS_EXTRAS:`	Monta o dicionário de feriados nacionais/estaduais, facultativos e adicionais.
245	`        f[pd.Timestamp(d).date()] = "Feriado extra"`	Monta o dicionário de feriados nacionais/estaduais, facultativos e adicionais.
246	`    return f`	Monta o dicionário de feriados nacionais/estaduais, facultativos e adicionais.
247	``	Linha em branco usada para separar blocos.
248	``	Linha em branco usada para separar blocos.
249	`def montar_calendario(ano: int) -> pd.DataFrame:`	Cria calendário diário do ano, identifica dia útil, feriado e semana iniciada na segunda-feira.
250	`    fer = feriados(ano)`	Cria calendário diário do ano, identifica dia útil, feriado e semana iniciada na segunda-feira.
251	`    datas = pd.date_range(f"{ano}-01-01", f"{ano}-12-31")`	Cria calendário diário do ano, identifica dia útil, feriado e semana iniciada na segunda-feira.

Linha	Código	Explicação
252	`    cal = pd.DataFrame({"DATA": datas})`	Cria calendário diário do ano, identifica dia útil, feriado e semana iniciada na segunda-feira.
253	`    cal["MES"] = cal["DATA"].dt.month`	Cria calendário diário do ano, identifica dia útil, feriado e semana iniciada na segunda-feira.
254	`    cal["DIA_SEMANA"] = cal["DATA"].dt.dayofweek.map(`	Cria calendário diário do ano, identifica dia útil, feriado e semana iniciada na segunda-feira.
255	`        {0: "segunda", 1: "terça", 2: "quarta", 3: "quinta", 4: "sexta", 5: "sábado", 6: "domingo"})`	Cria calendário diário do ano, identifica dia útil, feriado e semana iniciada na segunda-feira.
256	`    cal["FERIADO"] = cal["DATA"].dt.date.map(fer)`	Cria calendário diário do ano, identifica dia útil, feriado e semana iniciada na segunda-feira.
257	`    cal["DIA_UTIL"] = (cal["DATA"].dt.dayofweek < 5) & cal["FERIADO"].isna()`	Cria calendário diário do ano, identifica dia útil, feriado e semana iniciada na segunda-feira.
258	`    inicio = cal["DATA"] - pd.to_timedelta(cal["DATA"].dt.dayofweek, unit="D")`	Cria calendário diário do ano, identifica dia útil, feriado e semana iniciada na segunda-feira.
259	`    cal["SEMANA_INICIO"] = inicio.dt.strftime("%Y-%m-%d")`	Cria calendário diário do ano, identifica dia útil, feriado e semana iniciada na segunda-feira.

Linha	Código	Explicação
260	`    ordem = {s: i + 1 for i, s in enumerate(sorted(cal["SEMANA_INICIO"].unique()))}`	Cria calendário diário do ano, identifica dia útil, feriado e semana iniciada na segunda-feira.
261	`    cal["SEMANA"] = cal["SEMANA_INICIO"].map(ordem)`	Cria calendário diário do ano, identifica dia útil, feriado e semana iniciada na segunda-feira.
262	`    return cal`	Cria calendário diário do ano, identifica dia útil, feriado e semana iniciada na segunda-feira.
263	``	Linha em branco usada para separar blocos.
264	``	Linha em branco usada para separar blocos.
265	`def criar_modelo_ferias(caminho: str, fiscais: list, motoristas: list):`	Cria um modelo Excel de férias com abas de preenchimento e instruções para fiscais/motoristas.
266	`    linhas = [(p, "FISCAL") for p in fiscais] + [(p, "MOTORISTA") for p in motoristas]`	Cria um modelo Excel de férias com abas de preenchimento e instruções para fiscais/motoristas.
267	`    ferias = pd.DataFrame(linhas, columns=["PESSOA", "TIPO"])`	Cria um modelo Excel de férias com abas de preenchimento e instruções para fiscais/motoristas.

Linha	Código	Explicação
268	`    ferias["INICIO"] = pd.NaT`	Cria um modelo Excel de férias com abas de preenchimento e instruções para fiscais/motoristas.
269	`    ferias["FIM"] = pd.NaT`	Cria um modelo Excel de férias com abas de preenchimento e instruções para fiscais/motoristas.
270	`    ferias["OBSERVACAO"] = ""`	Cria um modelo Excel de férias com abas de preenchimento e instruções para fiscais/motoristas.
271	`    ajuda = pd.DataFrame({"Como preencher": [`	Cria um modelo Excel de férias com abas de preenchimento e instruções para fiscais/motoristas.
272	`        "1) Na aba Ferias, preencha INICIO e FIM (datas, no formato dd/mm/aaaa) de cada período de férias.",`	Cria um modelo Excel de férias com abas de preenchimento e instruções para fiscais/motoristas.
273	`        "2) Uma pessoa pode ter mais de uma linha (férias parceladas): copie a linha e mude as datas.",`	Cria um modelo Excel de férias com abas de preenchimento e instruções para fiscais/motoristas.
274	`        "3) Linhas com INICIO ou FIM em branco são ignoradas.",`	Cria um modelo Excel de férias com abas de preenchimento e instruções para fiscais/motoristas.
275	`        "4) Use o nome EXATAMENTE como está na coluna PESSOA (igual ao da base).",`	Cria um modelo Excel de férias com abas de preenchimento e instruções para fiscais/motoristas.

Linha	Código	Explicação
276	"5) Pode incluir também licenças, afastamentos e treinamentos: qualquer período sem expediente.",	Cria um modelo Excel de férias com abas de preenchimento e instruções para fiscais/motoristas.
277	"6) Salve com o nome Ferias_Fiscais_Motoristas.xlsx na mesma pasta e rode o programa de novo."]})	Cria um modelo Excel de férias com abas de preenchimento e instruções para fiscais/motoristas.
278	with pd.ExcelWriter(caminho, engine="openpyxl") as w:	Cria um modelo Excel de férias com abas de preenchimento e instruções para fiscais/motoristas.
279	ferias.to_excel(w, sheet_name="Ferias", index=False)	Cria um modelo Excel de férias com abas de preenchimento e instruções para fiscais/motoristas.
280	ajuda.to_excel(w, sheet_name="Como_preencher", index=False)	Cria um modelo Excel de férias com abas de preenchimento e instruções para fiscais/motoristas.
281	for ws in w.book.worksheets:	Cria um modelo Excel de férias com abas de preenchimento e instruções para fiscais/motoristas.
282	for col in ws.columns:	Cria um modelo Excel de férias com abas de preenchimento e instruções para fiscais/motoristas.
283	ws.column_dimensions[col[0].column_letter].width = 42 if ws.title == "Ferias" and col[0].column == 1 else 18	Cria um modelo Excel de férias com abas de preenchimento e instruções para fiscais/motoristas.

Linha	Código	Explicação
284	`        if ws.title == "Como_preencher":`	Cria um modelo Excel de férias com abas de preenchimento e instruções para fiscais/motoristas.
285	`            ws.column_dimensions["A"].width = 110`	Cria um modelo Excel de férias com abas de preenchimento e instruções para fiscais/motoristas.
286	``	Linha em branco usada para separar blocos.
287	``	Linha em branco usada para separar blocos.
288	`def ler_ferias(caminho: str, fiscais: list, motoristas: list) -> pd.DataFrame:`	Lê férias existentes, normaliza colunas/datas, descarta registros incompletos/desconhecidos e retorna apenas períodos válidos.
289	`    vazio = pd.DataFrame(columns=["PESSOA", "TIPO", "INICIO", "FIM", "OBSERVACAO"])`	Lê férias existentes, normaliza colunas/datas, descarta registros incompletos/desconhecidos e retorna apenas períodos válidos.
290	`    if not os.path.exists(caminho):`	Lê férias existentes, normaliza colunas/datas, descarta registros incompletos/desconhecidos e retorna apenas períodos válidos.

Linha	Código	Explicação
291	`        criar_modelo_ferias(caminho, fiscais, motoristas)`	Lê férias existentes, normaliza colunas/datas, descarta registros incompletos/desconhecidos e retorna apenas períodos válidos.
292	`        print(f"AVISO: não encontrei {os.path.basename(caminho)}. Criei um MODELO na pasta; preencha as férias e rode de novo.\n"`	Lê férias existentes, normaliza colunas/datas, descarta registros incompletos/desconhecidos e retorna apenas períodos válidos.
293	`                "       Enquanto isso, o plano considera que ninguém está de férias.")`	Lê férias existentes, normaliza colunas/datas, descarta registros incompletos/desconhecidos e retorna apenas períodos válidos.
294	`        return vazio`	Lê férias existentes, normaliza colunas/datas, descarta registros incompletos/desconhecidos e retorna apenas períodos válidos.
295	`    f = pd.read_excel(caminho, sheet_name="Ferias")`	Lê férias existentes, normaliza colunas/datas, descarta registros incompletos/desconhecidos e retorna apenas períodos válidos.
296	`    f.columns = [str(c).strip().upper() for c in f.columns]`	Lê férias existentes, normaliza colunas/datas, descarta registros incompletos/desconhecidos e retorna apenas períodos válidos.

Linha	Código	Explicação
297	`    for c in ("INICIO", "FIM"):`	Lê férias existentes, normaliza colunas/datas, descarta registros incompletos/desconhecidos e retorna apenas períodos válidos.
298	`        f[c] = pd.to_datetime(f[c], errors="coerce", dayfirst=True)`	Lê férias existentes, normaliza colunas/datas, descarta registros incompletos/desconhecidos e retorna apenas períodos válidos.
299	`    f["PESSOA"] = f["PESSOA"].astype(str).str.strip().str.upper()`	Lê férias existentes, normaliza colunas/datas, descarta registros incompletos/desconhecidos e retorna apenas períodos válidos.
300	`    f = f.dropna(subset=["INICIO", "FIM"])`	Lê férias existentes, normaliza colunas/datas, descarta registros incompletos/desconhecidos e retorna apenas períodos válidos.
301	`    if "TIPO" not in f.columns:`	Lê férias existentes, normaliza colunas/datas, descarta registros incompletos/desconhecidos e retorna apenas períodos válidos.
302	`        f["TIPO"] = ""`	Lê férias existentes, normaliza colunas/datas, descarta registros incompletos/desconhecidos e retorna apenas períodos válidos.

Linha	Código	Explicação
303	if "OBSERVACAO" not in f.columns:	Lê férias existentes, normaliza colunas/datas, descarta registros incompletos/desconhecidos e retorna apenas períodos válidos.
304	f["OBSERVACAO"] = ""	Lê férias existentes, normaliza colunas/datas, descarta registros incompletos/desconhecidos e retorna apenas períodos válidos.
305	conhecidas = set(fiscais) | set(motoristas)	Lê férias existentes, normaliza colunas/datas, descarta registros incompletos/desconhecidos e retorna apenas períodos válidos.
306	desconhecidas = sorted(set(f["PESSOA"]) - conhecidas)	Lê férias existentes, normaliza colunas/datas, descarta registros incompletos/desconhecidos e retorna apenas períodos válidos.
307	if desconhecidas:	Lê férias existentes, normaliza colunas/datas, descarta registros incompletos/desconhecidos e retorna apenas períodos válidos.
308	print("AVISO: nomes nas férias que não existem na base (ignorados):", desconhecidas)	Lê férias existentes, normaliza colunas/datas, descarta registros incompletos/desconhecidos e retorna apenas períodos válidos.

Linha	Código	Explicação
309	`    f = f[f["PESSOA"].isin(conhecidas)]`	Lê férias existentes, normaliza colunas/datas, descarta registros incompletos/desconhecidos e retorna apenas períodos válidos.
310	`    print(f"Férias lidas: {len(f)} períodos, {f['PESSOA'].nunique()} pessoas")`	Lê férias existentes, normaliza colunas/datas, descarta registros incompletos/desconhecidos e retorna apenas períodos válidos.
311	`    return f[["PESSOA", "TIPO", "INICIO", "FIM", "OBSERVACAO"]]`	Lê férias existentes, normaliza colunas/datas, descarta registros incompletos/desconhecidos e retorna apenas períodos válidos.
312	``	Linha em branco usada para separar blocos.
313	``	Linha em branco usada para separar blocos.
314	`def montar_disponibilidade(cal: pd.DataFrame, pessoas: list, ferias: pd.DataFrame) -> pd.DataFrame:`	Cria matriz de disponibilidade por dia útil e pessoa; marca como indisponíveis os dias dentro dos períodos de férias.
315	`    """Linhas = dias úteis; colunas = pessoas; True = disponível."""`	Cria matriz de disponibilidade por dia útil e pessoa; marca como indisponíveis os dias dentro dos períodos de férias.

Linha	Código	Explicação
316	`    uteis = pd.DatetimeIndex(cal.loc[cal["DIA_UTIL"], "DATA"])`	Cria matriz de disponibilidade por dia útil e pessoa; marca como indisponíveis os dias dentro dos períodos de férias.
317	`    disp = pd.DataFrame(True, index=uteis, columns=pessoas)`	Cria matriz de disponibilidade por dia útil e pessoa; marca como indisponíveis os dias dentro dos períodos de férias.
318	`    for _, r in ferias.iterrows():`	Cria matriz de disponibilidade por dia útil e pessoa; marca como indisponíveis os dias dentro dos períodos de férias.
319	`        if r["PESSOA"] in disp.columns:`	Cria matriz de disponibilidade por dia útil e pessoa; marca como indisponíveis os dias dentro dos períodos de férias.
320	`            disp.loc[(disp.index >= r["INICIO"]) & (disp.index <= r["FIM"]), r["PESSOA"]] = False`	Cria matriz de disponibilidade por dia útil e pessoa; marca como indisponíveis os dias dentro dos períodos de férias.
321	`    return disp`	Cria matriz de disponibilidade por dia útil e pessoa; marca como indisponíveis os dias dentro dos períodos de férias.
322	``	Linha em branco usada para separar blocos.
323	``	Linha em branco usada para separar blocos.

Linha	Código	Explicação
324	def formatar_planilhas(writer):	Formata planilhas: congela cabeçalho, aplica cabeçalho azul, percentuais e largura automática.
325	from openpyxl.styles import Font, PatternFill	Formata planilhas: congela cabeçalho, aplica cabeçalho azul, percentuais e largura automática.
326	for ws in writer.book.worksheets:	Formata planilhas: congela cabeçalho, aplica cabeçalho azul, percentuais e largura automática.
327	ws.freeze_panes = "B2"	Formata planilhas: congela cabeçalho, aplica cabeçalho azul, percentuais e largura automática.
328	for c in ws[1]:	Formata planilhas: congela cabeçalho, aplica cabeçalho azul, percentuais e largura automática.
329	c.font = Font(bold=True, color="FFFFFF")	Formata planilhas: congela cabeçalho, aplica cabeçalho azul, percentuais e largura automática.
330	c.fill = PatternFill("solid", fgColor="2E6F9E")	Formata planilhas: congela cabeçalho, aplica cabeçalho azul, percentuais e largura automática.
331	nome = str(c.value or "")	Formata planilhas: congela cabeçalho, aplica cabeçalho azul, percentuais e largura automática.
332	if nome.startswith("PCT_") or nome.endswith("_PCT"):	Formata planilhas: congela cabeçalho, aplica cabeçalho azul, percentuais e largura automática.

Linha	Código	Explicação
333	`            for cel in ws[c.column_letter][1:]:`	Formata planilhas: congela cabeçalho, aplica cabeçalho azul, percentuais e largura automática.
334	`                cel.number_format = "0.0%"`	Formata planilhas: congela cabeçalho, aplica cabeçalho azul, percentuais e largura automática.
335	`        for col in ws.columns:`	Formata planilhas: congela cabeçalho, aplica cabeçalho azul, percentuais e largura automática.
336	`            largura = max(len(str(c.value)) if c.value is not None else 0 for c in list(col)[:200])`	Formata planilhas: congela cabeçalho, aplica cabeçalho azul, percentuais e largura automática.
337	`            ws.column_dimensions[col[0].column_letter].width = min(max(10, largura + 2), 70)`	Formata planilhas: congela cabeçalho, aplica cabeçalho azul, percentuais e largura automática.
338	``	Linha em branco usada para separar blocos.
339	``	Linha em branco usada para separar blocos.
340	``	Linha em branco usada para separar blocos.
341	``	Linha em branco usada para separar blocos.

Linha	Código	Explicação
342	# ---------- LEITURA DA BASE GEOCODIFICADA ----------	Valida e lê a base geocodificada, corrige municípios, reatribui removidos, consolida visitas por equipe/dia/estabelecimento, identifica última visita e calcula retorno real ou estimado à sede.
343	COLUNAS_OBRIGATORIAS = ["FISCAL", "ID_EQUIPE", "COD_PROPR", "MUNICIPIO", "DT_HORA_VERIF", "CODITEM", "LATITUDE", "LONGITUDE",	Valida e lê a base geocodificada, corrige municípios, reatribui removidos, consolida visitas por equipe/dia/estabelecimento, identifica última visita e calcula retorno real ou estimado à sede.
344	"PRECISAO_GEO", "DISTANCIA_RODOVIARIA_ESTIMADA_KM", "TEMPO_DESLOCAMENTO_MIN"]	Valida e lê a base geocodificada, corrige municípios, reatribui removidos, consolida visitas por equipe/dia/estabelecimento, identifica última visita e calcula retorno real ou estimado à sede.
345	``	Linha em branco usada para separar blocos.
346	``	Linha em branco usada para separar blocos.
347	def ler_base(caminho: str):	Valida e lê a base geocodificada, corrige municípios, reatribui removidos, consolida visitas por equipe/dia/estabelecimento, identifica última visita e calcula retorno real ou estimado à sede.

Linha	Código	Explicação
348	`    df = pd.read_excel(caminho, sheet_name=0)`	Valida e lê a base geocodificada, corrige municípios, reatribui removidos, consolida visitas por equipe/dia/estabelecimento, identifica última visita e calcula retorno real ou estimado à sede.
349	`    falta = [c for c in COLUNAS_OBRIGATORIAS if c not in df.columns]`	Valida e lê a base geocodificada, corrige municípios, reatribui removidos, consolida visitas por equipe/dia/estabelecimento, identifica última visita e calcula retorno real ou estimado à sede.
350	`    if falta:`	Valida e lê a base geocodificada, corrige municípios, reatribui removidos, consolida visitas por equipe/dia/estabelecimento, identifica última visita e calcula retorno real ou estimado à sede.
351	`        raise RuntimeError(f"Colunas ausentes no {os.path.basename(caminho)}: {falta}")`	Valida e lê a base geocodificada, corrige municípios, reatribui removidos, consolida visitas por equipe/dia/estabelecimento, identifica última visita e calcula retorno real ou estimado à sede.

Linha	Código	Explicação
352	`    df["DT"] = pd.to_datetime(df["DT_HORA_VERIF"], format="%d/%m/%Y %H:%M:%S", errors="coerce")`	Valida e lê a base geocodificada, corrige municípios, reatribui removidos, consolida visitas por equipe/dia/estabelecimento, identifica última visita e calcula retorno real ou estimado à sede.
353	`    df = df.dropna(subset=["DT", "FISCAL"]).copy()`	Valida e lê a base geocodificada, corrige municípios, reatribui removidos, consolida visitas por equipe/dia/estabelecimento, identifica última visita e calcula retorno real ou estimado à sede.
354	`    df["DIA"] = df["DT"].dt.normalize()`	Valida e lê a base geocodificada, corrige municípios, reatribui removidos, consolida visitas por equipe/dia/estabelecimento, identifica última visita e calcula retorno real ou estimado à sede.
355	`    df["MUNICIPIO"] = df["MUNICIPIO"].replace(AJUSTES_NOME_MUN)`	Valida e lê a base geocodificada, corrige municípios, reatribui removidos, consolida visitas por equipe/dia/estabelecimento, identifica última visita e calcula retorno real ou estimado à sede.

Linha	Código	Explicação
356	`    if "ESPECIE" not in df.columns:`	Valida e lê a base geocodificada, corrige municípios, reatribui removidos, consolida visitas por equipe/dia/estabelecimento, identifica última visita e calcula retorno real ou estimado à sede.
357	`        df["ESPECIE"] = "N/D"`	Valida e lê a base geocodificada, corrige municípios, reatribui removidos, consolida visitas por equipe/dia/estabelecimento, identifica última visita e calcula retorno real ou estimado à sede.
358	`    df = reatribuir_fiscais_removidos(df, FISCAIS_REMOVER)`	Valida e lê a base geocodificada, corrige municípios, reatribui removidos, consolida visitas por equipe/dia/estabelecimento, identifica última visita e calcula retorno real ou estimado à sede.
359	`    ret_km = "DISTANCIA_RETORNO_SEDE_KM" in df.columns and "TEMPO_RETORNO_SEDE_MIN" in df.columns`	Valida e lê a base geocodificada, corrige municípios, reatribui removidos, consolida visitas por equipe/dia/estabelecimento, identifica última visita e calcula retorno real ou estimado à sede.

Linha	Código	Explicação
360	agg = dict(FISCAL=("FISCAL", "first"), FISCAL_ORIG=("FISCAL_ORIG", "first"), HORA_INICIO=("DT", "min"), NOMEFANTASIA=("NOMEFANTASIA", "first"),	Valida e lê a base geocodificada, corrige municípios, reatribui removidos, consolida visitas por equipe/dia/estabelecimento, identifica última visita e calcula retorno real ou estimado à sede.
361	ENDERECO=("ENDERECO_PADRONIZADO", "first"), MUNICIPIO=("MUNICIPIO", "first"),	Valida e lê a base geocodificada, corrige municípios, reatribui removidos, consolida visitas por equipe/dia/estabelecimento, identifica última visita e calcula retorno real ou estimado à sede.
362	BAIRRO=("BAIRRO", "first"), LATITUDE=("LATITUDE", "first"), LONGITUDE=("LONGITUDE", "first"),	Valida e lê a base geocodificada, corrige municípios, reatribui removidos, consolida visitas por equipe/dia/estabelecimento, identifica última visita e calcula retorno real ou estimado à sede.
363	PRECISAO_GEO=("PRECISAO_GEO", "first"), ITENS=("CODITEM", "size"),	Valida e lê a base geocodificada, corrige municípios, reatribui removidos, consolida visitas por equipe/dia/estabelecimento, identifica última visita e calcula retorno real ou estimado à sede.

Linha	Código	Explicação
364	KM_TRECHO=("DISTANCIA_RODOVIARIA_ESTIMADA_KM", "first"), MIN_TRECHO=("TEMPO_DESLOCAMENTO_MIN", "first"))	Valida e lê a base geocodificada, corrige municípios, reatribui removidos, consolida visitas por equipe/dia/estabelecimento, identifica última visita e calcula retorno real ou estimado à sede.
365	if ret_km:	Valida e lê a base geocodificada, corrige municípios, reatribui removidos, consolida visitas por equipe/dia/estabelecimento, identifica última visita e calcula retorno real ou estimado à sede.
366	agg.update(KM_RET_BASE=("DISTANCIA_RETORNO_SEDE_KM", "first"), MIN_RET_BASE=("TEMPO_RETORNO_SEDE_MIN", "first"))	Valida e lê a base geocodificada, corrige municípios, reatribui removidos, consolida visitas por equipe/dia/estabelecimento, identifica última visita e calcula retorno real ou estimado à sede.
367	vis = (df.groupby(["ID_EQUIPE", "DIA", "COD_PROPR"], sort=False).agg(**agg).reset_index()	Valida e lê a base geocodificada, corrige municípios, reatribui removidos, consolida visitas por equipe/dia/estabelecimento, identifica última visita e calcula retorno real ou estimado à sede.

Linha	Código	Explicação
368	.sort_values(["ID_EQUIPE", "DIA", "HORA_INICIO", "COD_PROPR"]).reset_index(drop=True))	Valida e lê a base geocodificada, corrige municípios, reatribui removidos, consolida visitas por equipe/dia/estabelecimento, identifica última visita e calcula retorno real ou estimado à sede.
369	g = vis.groupby(["ID_EQUIPE", "DIA"], sort=False)	Valida e lê a base geocodificada, corrige municípios, reatribui removidos, consolida visitas por equipe/dia/estabelecimento, identifica última visita e calcula retorno real ou estimado à sede.
370	vis["SEQ"] = g.cumcount() + 1	Valida e lê a base geocodificada, corrige municípios, reatribui removidos, consolida visitas por equipe/dia/estabelecimento, identifica última visita e calcula retorno real ou estimado à sede.
371	vis["N"] = g["COD_PROPR"].transform("size")	Valida e lê a base geocodificada, corrige municípios, reatribui removidos, consolida visitas por equipe/dia/estabelecimento, identifica última visita e calcula retorno real ou estimado à sede.

Linha	Código	Explicação
372	`    vis["GEO_BAIXA"] = vis["PRECISAO_GEO"].isna() | vis["PRECISAO_GEO"].isin(GEO_BAIXA)`	Valida e lê a base geocodificada, corrige municípios, reatribui removidos, consolida visitas por equipe/dia/estabelecimento, identifica última visita e calcula retorno real ou estimado à sede.
373	`    ultima = vis["SEQ"] == vis["N"]`	Valida e lê a base geocodificada, corrige municípios, reatribui removidos, consolida visitas por equipe/dia/estabelecimento, identifica última visita e calcula retorno real ou estimado à sede.
374	`    if ret_km:`	Valida e lê a base geocodificada, corrige municípios, reatribui removidos, consolida visitas por equipe/dia/estabelecimento, identifica última visita e calcula retorno real ou estimado à sede.
375	`        vis["KM_RET"] = vis["KM_RET_BASE"].where(ultima, 0.0)`	Valida e lê a base geocodificada, corrige municípios, reatribui removidos, consolida visitas por equipe/dia/estabelecimento, identifica última visita e calcula retorno real ou estimado à sede.

Linha	Código	Explicação
376	`        vis["MIN_RET"] = vis["MIN_RET_BASE"].where(ultima, 0.0)`	Valida e lê a base geocodificada, corrige municípios, reatribui removidos, consolida visitas por equipe/dia/estabelecimento, identifica última visita e calcula retorno real ou estimado à sede.
377	`        fonte_ret = "colunas de retorno da base"`	Valida e lê a base geocodificada, corrige municípios, reatribui removidos, consolida visitas por equipe/dia/estabelecimento, identifica última visita e calcula retorno real ou estimado à sede.
378	`    else:`	Valida e lê a base geocodificada, corrige municípios, reatribui removidos, consolida visitas por equipe/dia/estabelecimento, identifica última visita e calcula retorno real ou estimado à sede.
379	`        d = haversine_km(vis["LATITUDE"], vis["LONGITUDE"], SEDE[0], SEDE[1]) * FATOR_CIRCUIDADE`	Valida e lê a base geocodificada, corrige municípios, reatribui removidos, consolida visitas por equipe/dia/estabelecimento, identifica última visita e calcula retorno real ou estimado à sede.

Linha	Código	Explicação
380	`    vis["KM_RET"] = d.where(ultima, 0.0)`	Valida e lê a base geocodificada, corrige municípios, reatribui removidos, consolida visitas por equipe/dia/estabelecimento, identifica última visita e calcula retorno real ou estimado à sede.
381	`    vis["MIN_RET"] = (d * 60 / VELOCIDADE_KMH).where(ultima, 0.0)`	Valida e lê a base geocodificada, corrige municípios, reatribui removidos, consolida visitas por equipe/dia/estabelecimento, identifica última visita e calcula retorno real ou estimado à sede.
382	`    fonte_ret = f"estimado (linha reta x {FATOR_CIRCUIDADE} a {VELOCIDADE_KMH:g} km/h até a central)"`	Valida e lê a base geocodificada, corrige municípios, reatribui removidos, consolida visitas por equipe/dia/estabelecimento, identifica última visita e calcula retorno real ou estimado à sede.
383	`  return df, vis, fonte_ret`	Valida e lê a base geocodificada, corrige municípios, reatribui removidos, consolida visitas por equipe/dia/estabelecimento, identifica última visita e calcula retorno real ou estimado à sede.
384	``	Linha em branco usada para separar blocos.
385	``	Linha em branco usada para separar blocos.

Linha	Código	Explicação
386	# ---------- TEMPO DE FISCALIZAÇÃO POR FAIXA DE ITENS ----------	Define faixas de quantidade de itens por visita e classifica cada estabelecimento.
387	FAIXAS = [("1 item", 1, 1), ("2 a 3 itens", 2, 3), ("4 a 6 itens", 4, 6), ("7+ itens", 7, 10 ** 6)]	Define faixas de quantidade de itens por visita e classifica cada estabelecimento.
388	``	Linha em branco usada para separar blocos.
389	``	Linha em branco usada para separar blocos.
390	def faixa_de(itens: pd.Series) -> pd.Series:	Define faixas de quantidade de itens por visita e classifica cada estabelecimento.
391	out = pd.Series(index=itens.index, dtype=object)	Define faixas de quantidade de itens por visita e classifica cada estabelecimento.
392	for nome, lo, hi in FAIXAS:	Define faixas de quantidade de itens por visita e classifica cada estabelecimento.
393	out[(itens >= lo) & (itens <= hi)] = nome	Define faixas de quantidade de itens por visita e classifica cada estabelecimento.
394	return out	Define faixas de quantidade de itens por visita e classifica cada estabelecimento.
395	``	Linha em branco usada para separar blocos.

Linha	Código	Explicação
396	``	Linha em branco usada para separar blocos.
397	def calibrar_tempo(vis: pd.DataFrame):	Calibra o tempo de fiscalização a partir do intervalo entre visitas consecutivas, descontando deslocamento, filtrando intervalos inválidos e calculando médias por faixa de itens.
398	"""Tempo na visita = intervalo até a próxima visita da equipe - deslocamento até ela. Média por faixa de itens."""	Calibra o tempo de fiscalização a partir do intervalo entre visitas consecutivas, descontando deslocamento, filtrando intervalos inválidos e calculando médias por faixa de itens.
399	g = vis.groupby(["ID_EQUIPE", "DIA"], sort=False)	Calibra o tempo de fiscalização a partir do intervalo entre visitas consecutivas, descontando deslocamento, filtrando intervalos inválidos e calculando médias por faixa de itens.
400	delta = (g["HORA_INICIO"].shift(-1) - vis["HORA_INICIO"]).dt.total_seconds() / 60	Calibra o tempo de fiscalização a partir do intervalo entre visitas consecutivas, descontando deslocamento, filtrando intervalos inválidos e calculando médias por faixa de itens.

Linha	Código	Explicação
401	`    viagem = g["MIN_TRECHO"].shift(-1)`	Calibra o tempo de fiscalização a partir do intervalo entre visitas consecutivas, descontando deslocamento, filtrando intervalos inválidos e calculando médias por faixa de itens.
402	`    prox_ok = g["GEO_BAIXA"].shift(-1).eq(False)`	Calibra o tempo de fiscalização a partir do intervalo entre visitas consecutivas, descontando deslocamento, filtrando intervalos inválidos e calculando médias por faixa de itens.
403	`    obs = delta - viagem`	Calibra o tempo de fiscalização a partir do intervalo entre visitas consecutivas, descontando deslocamento, filtrando intervalos inválidos e calculando médias por faixa de itens.
404	`    tem = delta.notna()`	Calibra o tempo de fiscalização a partir do intervalo entre visitas consecutivas, descontando deslocamento, filtrando intervalos inválidos e calculando médias por faixa de itens.
405	`    m_int = tem & (delta <= LIMITE_INTERVALO_MIN)`	Calibra o tempo de fiscalização a partir do intervalo entre visitas consecutivas, descontando deslocamento, filtrando intervalos inválidos e calculando médias por faixa de itens.

Linha	Código	Explicação
406	`    m_coord = m_int & viagem.notna()`	Calibra o tempo de fiscalização a partir do intervalo entre visitas consecutivas, descontando deslocamento, filtrando intervalos inválidos e calculando médias por faixa de itens.
407	`    m_geo = m_coord & (~vis["GEO_BAIXA"]) & prox_ok`	Calibra o tempo de fiscalização a partir do intervalo entre visitas consecutivas, descontando deslocamento, filtrando intervalos inválidos e calculando médias por faixa de itens.
408	`    if int((m_geo & (obs >= 0)).sum()) < MIN_PARES_CALIBRACAO:`	Calibra o tempo de fiscalização a partir do intervalo entre visitas consecutivas, descontando deslocamento, filtrando intervalos inválidos e calculando médias por faixa de itens.
409	`        print("AVISO: poucos pares com coordenada precisa; a calibração usará todos os pares com deslocamento conhecido.")`	Calibra o tempo de fiscalização a partir do intervalo entre visitas consecutivas, descontando deslocamento, filtrando intervalos inválidos e calculando médias por faixa de itens.
410	`        m_geo = m_coord`	Calibra o tempo de fiscalização a partir do intervalo entre visitas consecutivas, descontando deslocamento, filtrando intervalos inválidos e calculando médias por faixa de itens.

Linha	Código	Explicação
411	`    m_pos = m_geo & (obs >= 0)`	Calibra o tempo de fiscalização a partir do intervalo entre visitas consecutivas, descontando deslocamento, filtrando intervalos inválidos e calculando médias por faixa de itens.
412	`    val = obs[m_pos]`	Calibra o tempo de fiscalização a partir do intervalo entre visitas consecutivas, descontando deslocamento, filtrando intervalos inválidos e calculando médias por faixa de itens.
413	`    if len(val) == 0:`	Calibra o tempo de fiscalização a partir do intervalo entre visitas consecutivas, descontando deslocamento, filtrando intervalos inválidos e calculando médias por faixa de itens.
414	`        raise RuntimeError("Nenhum par de visitas válido para calibrar o tempo de fiscalização.")`	Calibra o tempo de fiscalização a partir do intervalo entre visitas consecutivas, descontando deslocamento, filtrando intervalos inválidos e calculando médias por faixa de itens.
415	`    media = float(val.mean())`	Calibra o tempo de fiscalização a partir do intervalo entre visitas consecutivas, descontando deslocamento, filtrando intervalos inválidos e calculando médias por faixa de itens.

Linha	Código	Explicação
416	`    fx = faixa_de(vis.loc[m_pos, "ITENS"])`	Calibra o tempo de fiscalização a partir do intervalo entre visitas consecutivas, descontando deslocamento, filtrando intervalos inválidos e calculando médias por faixa de itens.
417	`    linhas = []`	Calibra o tempo de fiscalização a partir do intervalo entre visitas consecutivas, descontando deslocamento, filtrando intervalos inválidos e calculando médias por faixa de itens.
418	`    tempos = {}`	Calibra o tempo de fiscalização a partir do intervalo entre visitas consecutivas, descontando deslocamento, filtrando intervalos inválidos e calculando médias por faixa de itens.
419	`    for nome, lo, hi in FAIXAS:`	Calibra o tempo de fiscalização a partir do intervalo entre visitas consecutivas, descontando deslocamento, filtrando intervalos inválidos e calculando médias por faixa de itens.
420	`        v = val[fx == nome]`	Calibra o tempo de fiscalização a partir do intervalo entre visitas consecutivas, descontando deslocamento, filtrando intervalos inválidos e calculando médias por faixa de itens.

Linha	Código	Explicação
421	usa = len(v) >= MIN_PARES_FAIXA	Calibra o tempo de fiscalização a partir do intervalo entre visitas consecutivas, descontando deslocamento, filtrando intervalos inválidos e calculando médias por faixa de itens.
422	tempos[nome] = float(v.mean()) if usa else media	Calibra o tempo de fiscalização a partir do intervalo entre visitas consecutivas, descontando deslocamento, filtrando intervalos inválidos e calculando médias por faixa de itens.
423	linhas.append({"FAIXA_ITENS": nome, "PARES": len(v), "MEDIA_MIN": float(v.mean()) if len(v) else np.nan,	Calibra o tempo de fiscalização a partir do intervalo entre visitas consecutivas, descontando deslocamento, filtrando intervalos inválidos e calculando médias por faixa de itens.
424	"MEDIANA_MIN": float(v.median()) if len(v) else np.nan,	Calibra o tempo de fiscalização a partir do intervalo entre visitas consecutivas, descontando deslocamento, filtrando intervalos inválidos e calculando médias por faixa de itens.
425	"TEMPO_USADO_MIN": tempos[nome], "OBSERVACAO": "" if usa else f"poucos pares (<{MIN_PARES_FAIXA}): usa a média geral"})	Calibra o tempo de fiscalização a partir do intervalo entre visitas consecutivas, descontando deslocamento, filtrando intervalos inválidos e calculando médias por faixa de itens.

Linha	Código	Explicação
426	`    if TEMPO_FISCALIZACAO_MANUAL_MIN is not None:`	Calibra o tempo de fiscalização a partir do intervalo entre visitas consecutivas, descontando deslocamento, filtrando intervalos inválidos e calculando médias por faixa de itens.
427	`        tempos = {k: float(TEMPO_FISCALIZACAO_MANUAL_MIN) for k in tempos}`	Calibra o tempo de fiscalização a partir do intervalo entre visitas consecutivas, descontando deslocamento, filtrando intervalos inválidos e calculando médias por faixa de itens.
428	`    resumo = pd.DataFrame([`	Calibra o tempo de fiscalização a partir do intervalo entre visitas consecutivas, descontando deslocamento, filtrando intervalos inválidos e calculando médias por faixa de itens.
429	`        ("Pares de visitas consecutivas no mesmo dia", int(tem.sum())),`	Calibra o tempo de fiscalização a partir do intervalo entre visitas consecutivas, descontando deslocamento, filtrando intervalos inválidos e calculando médias por faixa de itens.
430	`        ("Descartados: intervalo > limite", int((tem & ~m_int).sum())),`	Calibra o tempo de fiscalização a partir do intervalo entre visitas consecutivas, descontando deslocamento, filtrando intervalos inválidos e calculando médias por faixa de itens.

Linha	Código	Explicação
431	("Descartados: sem deslocamento conhecido", int((m_int & ~m_coord).sum())),	Calibra o tempo de fiscalização a partir do intervalo entre visitas consecutivas, descontando deslocamento, filtrando intervalos inválidos e calculando médias por faixa de itens.
432	("Descartados: geocodificação imprecisa", int((m_coord & ~m_geo).sum())),	Calibra o tempo de fiscalização a partir do intervalo entre visitas consecutivas, descontando deslocamento, filtrando intervalos inválidos e calculando médias por faixa de itens.
433	("Descartados: deslocamento maior que o intervalo", int((m_geo & ~m_pos).sum())),	Calibra o tempo de fiscalização a partir do intervalo entre visitas consecutivas, descontando deslocamento, filtrando intervalos inválidos e calculando médias por faixa de itens.
434	("Pares válidos", int(len(val))),	Calibra o tempo de fiscalização a partir do intervalo entre visitas consecutivas, descontando deslocamento, filtrando intervalos inválidos e calculando médias por faixa de itens.
435	("Tempo médio na visita (min)", media), ("Mediana (min)", float(val.median())),	Calibra o tempo de fiscalização a partir do intervalo entre visitas consecutivas, descontando deslocamento, filtrando intervalos inválidos e calculando médias por faixa de itens.

Linha	Código	Explicação
436	("Percentil 90 (min)", float(val.quantile(0.9)))], columns=["Indicador", "Valor"])	Calibra o tempo de fiscalização a partir do intervalo entre visitas consecutivas, descontando deslocamento, filtrando intervalos inválidos e calculando médias por faixa de itens.
437	return tempos, pd.DataFrame(linhas), resumo	Calibra o tempo de fiscalização a partir do intervalo entre visitas consecutivas, descontando deslocamento, filtrando intervalos inválidos e calculando médias por faixa de itens.
438	``	Linha em branco usada para separar blocos.
439	``	Linha em branco usada para separar blocos.
440	def aplicar_tempos(vis: pd.DataFrame, tempos: dict) -> pd.DataFrame:	Aplica tempos calibrados e calcula horas de fiscalização, horas de deslocamento, quilômetros e carga total por visita.
441	vis = vis.copy()	Aplica tempos calibrados e calcula horas de fiscalização, horas de deslocamento, quilômetros e carga total por visita.
442	vis["FAIXA_ITENS"] = faixa_de(vis["ITENS"])	Aplica tempos calibrados e calcula horas de fiscalização, horas de deslocamento, quilômetros e carga total por visita.

Linha	Código	Explicação
443	`    vis["H_FISC"] = vis["FAIXA_ITENS"].map(tempos) / 60.0`	Aplica tempos calibrados e calcula horas de fiscalização, horas de deslocamento, quilômetros e carga total por visita.
444	`    vis["H_DESLOC"] = (vis["MIN_TRECHO"].fillna(0.0) + vis["MIN_RET"].fillna(0.0)) / 60.0`	Aplica tempos calibrados e calcula horas de fiscalização, horas de deslocamento, quilômetros e carga total por visita.
445	`    vis["KM"] = vis["KM_TRECHO"].fillna(0.0) + vis["KM_RET"].fillna(0.0)`	Aplica tempos calibrados e calcula horas de fiscalização, horas de deslocamento, quilômetros e carga total por visita.
446	`    vis["H_TOTAL"] = vis["H_FISC"] + vis["H_DESLOC"]`	Aplica tempos calibrados e calcula horas de fiscalização, horas de deslocamento, quilômetros e carga total por visita.
447	`    return vis`	Aplica tempos calibrados e calcula horas de fiscalização, horas de deslocamento, quilômetros e carga total por visita.
448	``	Linha em branco usada para separar blocos.
449	``	Linha em branco usada para separar blocos.
450	`def historico_dia(vis: pd.DataFrame) -> pd.DataFrame:`	Agrupa a carga histórica por fiscal e dia.
451	`    """Uma linha por fiscal e dia (carga real do período)."""`	Agrupa a carga histórica por fiscal e dia.

Linha	Código	Explicação
452	`    d = (vis.groupby(["FISCAL", "DIA"])`	Agrupa a carga histórica por fiscal e dia.
453	`        .agg(LOCAIS=("COD_PROPR", "size"), ITENS=("ITENS", "sum"), H_FISCALIZACAO=("H_FISC", "sum"),`	Agrupa a carga histórica por fiscal e dia.
454	`             H_DESLOC=("H_DESLOC", "sum"), H_TOTAL=("H_TOTAL", "sum"), KM=("KM", "sum")).reset_index()`	Agrupa a carga histórica por fiscal e dia.
455	`        .rename(columns={"FISCAL": "PESSOA"}))`	Agrupa a carga histórica por fiscal e dia.
456	`    d["SAIDAS"] = 1`	Agrupa a carga histórica por fiscal e dia.
457	`    return d`	Agrupa a carga histórica por fiscal e dia.
458	``	Linha em branco usada para separar blocos.
459	``	Linha em branco usada para separar blocos.
460	`# ---------------------------- DEMANDA EM SAÍDAS POR MUNICÍPIO ----------------------------`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.

Linha	Código	Explicação
461	def montar_demanda(vis: pd.DataFrame) -> pd.DataFrame:	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.
462	prec = vis[~vis["GEO_BAIXA"]]	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.
463	g = vis.groupby("MUNICIPIO")	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.
464	dm = pd.DataFrame({"VISITAS": g.size(), "ESTABELECIMENTOS": g["COD_PROPR"].nunique(), "ITENS": g["ITENS"].sum(),	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.

Linha	Código	Explicação
465	"H_FISC_TOTAL": g["H_FISC"].sum()})	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.
466	dm["DIAS_COM_VISITA"] = vis.assign(FD=vis["FISCAL_ORIG"] + "|" + vis["DIA"].dt.strftime("%Y-%m-%d")).groupby("MUNICIPIO")["FD"].nunique()	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.
467	dm["LAT"] = prec.groupby("MUNICIPIO")["LATITUDE"].mean()	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.
468	dm["LON"] = prec.groupby("MUNICIPIO")["LONGITUDE"].mean()	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.

Linha	Código	Explicação
469	`    dm["LAT"] = dm["LAT"].fillna(g["LATITUDE"].mean())`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.
470	`    dm["LON"] = dm["LON"].fillna(g["LONGITUDE"].mean())`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.
471	`    dm["PCT_GEO_BAIXA_PRECISAO"] = g["GEO_BAIXA"].mean()`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.
472	``	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.

Linha	Código	Explicação
473	`    # viagem central <-> município: média dos primeiros trechos do dia (ida) e do retorno; reserva = estimativa pela distância`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.
474	`    prim = prec[prec["SEQ"] == 1].groupby("MUNICIPIO").agg(N1=("KM_TRECHO", "size"), IDA_KM=("KM_TRECHO", "mean"), IDA_MIN=("MIN_TRECHO", "mean"))`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.
475	`    dm = dm.join(prim)`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.
476	`    ult = prec[(prec["SEQ"] == prec["N"]) & (prec["KM_RET"] > 0)].groupby("MUNICIPIO").agg(`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.

Linha	Código	Explicação
477	`    N2=("KM_RET", "size"), VOLTA_KM=("KM_RET", "mean"), VOLTA_MIN=("MIN_RET", "mean"))`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.
478	`  dm = dm.join(ult)`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.
479	`  reta = haversine_km(dm["LAT"], dm["LON"], SEDE[0], SEDE[1])`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.
480	`  ok_i, ok_v = dm["N1"].fillna(0) >= MIN_AMOSTRA_VIAGEM, dm["N2"].fillna(0) >= MIN_AMOSTRA_VIAGEM`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.

Linha	Código	Explicação
481	`    # reserva: circuidade e velocidade medidas nos municípios que têm rota real na base`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.
482	`    base_ok = ok_i & (reta > 5)`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.
483	`    circ = float((dm.loc[base_ok, "IDA_KM"] / reta[base_ok]).median()) if base_ok.any() else FATOR_CIRCUIDADE`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.
484	`    vel = float(dm.loc[ok_i, "IDA_KM"].sum() / (dm.loc[ok_i, "IDA_MIN"].sum() / 60.0)) if ok_i.any() else VELOCIDADE_KMH`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.

Linha	Código	Explicação
485	`    est_km = reta * circ`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.
486	`    est_min = est_km * 60 / vel`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.
487	`    dm["IDA_KM"], dm["IDA_MIN"] = dm["IDA_KM"].where(ok_i, est_km), dm["IDA_MIN"].where(ok_i, est_min)`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.
488	`    dm["VOLTA_KM"], dm["VOLTA_MIN"] = dm["VOLTA_KM"].where(ok_v, dm["IDA_KM"]), dm["VOLTA_MIN"].where(ok_v, dm["IDA_MIN"])`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.

Linha	Código	Explicação
489	`    dm.attrs["circuidade_base"], dm.attrs["velocidade_base"] = circ, vel`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.
490	`    dm["FONTE_VIAGEM"] = np.where(ok_i, "rota da base (1ºs trechos do dia)", "estimada (distância x circuidade e velocidade da base)")`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.
491	`    dm["R_H_CHEIA"] = (dm["IDA_MIN"] + dm["VOLTA_MIN"]) / 60.0 # viagem de uma saída só nesse município`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.
492	`    dm["R_KM_CHEIA"] = dm["IDA_KM"] + dm["VOLTA_KM"]`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.

Linha	Código	Explicação
493	`    dias_trab = vis.groupby(["FISCAL_ORIG", "DIA"]).ngroups`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.
494	`    comp = dias_trab / dm["DIAS_COM_VISITA"].sum() if AJUSTAR_COMPARTILHAMENTO else 1.0`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.
495	`    dm.attrs["compartilhamento"] = float(comp)`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.
496	`    dm["R_H"] = dm["R_H_CHEIA"] * comp                    # custo médio por saída (parte da viagem é dividida)`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.

Linha	Código	Explicação
497	`    dm["R_KM"] = dm["R_KM_CHEIA"] * comp`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.
498	```	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.
499	`    # deslocamento entre visitas dentro do município`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.
500	`    vis = vis.copy()`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.

Linha	Código	Explicação
501	`    vis["MUN_ANT"] = vis.groupby(["ID_EQUIPE", "DIA"], sort=False)["MUNICIPIO"].shift()`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.
502	`    intra = vis[(vis["SEQ"] > 1) & (vis["MUN_ANT"] == vis["MUNICIPIO"]) & vis["MIN_TRECHO"].notna()]`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.
503	`    leg = intra.groupby("MUNICIPIO").agg(LEG_MIN=("MIN_TRECHO", "mean"), LEG_KM=("KM_TRECHO", "mean"))`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.
504	`    dm = dm.join(leg)`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.

Linha	Código	Explicação
505	`    dm["LEG_MIN"] = dm["LEG_MIN"].fillna(leg["LEG_MIN"].median())`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.
506	`    dm["LEG_KM"] = dm["LEG_KM"].fillna(leg["LEG_KM"].median())`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.
507	```	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.
508	`    # saídas`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.

Linha	Código	Explicação
509	`    dm["FATOR"] = FATOR_DEMANDA`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.
510	`    dm["DEMANDA_VISITAS"] = np.round(dm["VISITAS"] * FATOR_DEMANDA).astype(int)`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.
511	`    tf_m = dm["H_FISC_TOTAL"] / dm["VISITAS"]`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.
512	`    dm["TF_MEDIO_H"] = tf_m`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.

Linha	Código	Explicação
513	`    dm["VISITAS_POR_SAIDA_HIST"] = dm["VISITAS"] / dm["DIAS_COM_VISITA"]`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.
514	`    cabem = np.floor((JORNADA_MAX_H - dm["R_H_CHEIA"]) / (tf_m + dm["LEG_MIN"] / 60.0)).clip(lower=1)`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.
515	`    dm["VISITAS_POR_SAIDA"] = np.minimum(np.round(dm["VISITAS_POR_SAIDA_HIST"]).clip(lower=1), cabem).astype(int)`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.
516	`    dm["SAIDAS"] = np.ceil(dm["DEMANDA_VISITAS"] / dm["VISITAS_POR_SAIDA"]).astype(int)`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.

Linha	Código	Explicação
517	`    dm["S_LOCAIS"] = dm["DEMANDA_VISITAS"] / dm["SAIDAS"]`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.
518	`    dm["S_ITENS"] = dm["ITENS"] * FATOR_DEMANDA / dm["SAIDAS"]`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.
519	`    extra = (dm["S_LOCAIS"] - 1).clip(lower=0)`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.
520	`    dm["S_H_FISC"] = dm["H_FISC_TOTAL"] * FATOR_DEMANDA / dm["SAIDAS"]`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.

Linha	Código	Explicação
521	`    dm["S_H_DESLOC"] = dm["R_H"] + extra * dm["LEG_MIN"] / 60.0`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.
522	`    dm["S_H_TOTAL"] = dm["S_H_FISC"] + dm["S_H_DESLOC"]`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.
523	`    dm["S_KM"] = dm["R_KM"] + extra * dm["LEG_KM"]`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.
524	`    dm["SAIDA_EXCEDE_JORNADA"] = (dm["S_H_TOTAL"] - dm["R_H"] + dm["R_H_CHEIA"]) > JORNADA_MAX_H + 1e-9`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.

Linha	Código	Explicação
525	`    dm["REVISAR_DESLOCAMENTO"] = dm["PCT_GEO_BAIXA_PRECISAO"] >= 0.9`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.
526	`    dm["H_TOTAL_PLANO"] = dm["SAIDAS"] * dm["S_H_TOTAL"]`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.
527	`    dm["KM_PLANO"] = dm["SAIDAS"] * dm["S_KM"]`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.
528	`    out = dm.reset_index()`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.

Linha	Código	Explicação
529	`    # guarda os valores usados para aparecerem na aba Parametros (os atributos se perdem nos joins acima)`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.
530	`    out.attrs.update({"circuidade_base": float(circ), "velocidade_base": float(vel), "compartilhamento": float(comp)})`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.
531	`    return out`	Consolida a demanda por município, estima viagens de ida/volta e deslocamentos internos, converte visitas em saídas, calcula capacidade de jornada, indicadores por saída e sinalizações de revisão.
532	``	Linha em branco usada para separar blocos.
533	``	Linha em branco usada para separar blocos.
534	`def montar_zonas(dm: pd.DataFrame) -> pd.DataFrame:`	Agrupa municípios em zonas geográficas usando clustering hierárquico Ward e nomeia as zonas pelos municípios de maior demanda.

Linha	Código	Explicação
535	`    lat, lon = dm["LAT"].to_numpy(), dm["LON"].to_numpy()`	Agrupa municípios em zonas geográficas usando clustering hierárquico Ward e nomeia as zonas pelos municípios de maior demanda.
536	`    xy = np.column_stack([(lon - lon.mean()) * 111.0 * np.cos(np.radians(lat.mean())), (lat - lat.mean()) * 111.0])`	Agrupa municípios em zonas geográficas usando clustering hierárquico Ward e nomeia as zonas pelos municípios de maior demanda.
537	`    z = fcluster(linkage(xy, method="ward"), t=min(N_ZONAS, len(dm)), criterion="maxclust")`	Agrupa municípios em zonas geográficas usando clustering hierárquico Ward e nomeia as zonas pelos municípios de maior demanda.
538	`    zonas = pd.DataFrame({"MUNICIPIO": dm["MUNICIPIO"].values, "ZONA_ID": z, "LAT": lat, "LON": lon,`	Agrupa municípios em zonas geográficas usando clustering hierárquico Ward e nomeia as zonas pelos municípios de maior demanda.
539	`                          "DEMANDA_VISITAS": dm["DEMANDA_VISITAS"].values})`	Agrupa municípios em zonas geográficas usando clustering hierárquico Ward e nomeia as zonas pelos municípios de maior demanda.
540	`    nomes = {}`	Agrupa municípios em zonas geográficas usando clustering hierárquico Ward e nomeia as zonas pelos municípios de maior demanda.

Linha	Código	Explicação
541	`    for zid, g in zonas.groupby("ZONA_ID"):`	Agrupa municípios em zonas geográficas usando clustering hierárquico Ward e nomeia as zonas pelos municípios de maior demanda.
542	`        top = g.sort_values("DEMANDA_VISITAS", ascending=False)["MUNICIPIO"].head(2).str.title().tolist()`	Agrupa municípios em zonas geográficas usando clustering hierárquico Ward e nomeia as zonas pelos municípios de maior demanda.
543	`        nomes[zid] = f"Z{zid}: " + " / ".join(top)`	Agrupa municípios em zonas geográficas usando clustering hierárquico Ward e nomeia as zonas pelos municípios de maior demanda.
544	`    zonas["ZONA"] = zonas["ZONA_ID"].map(nomes)`	Agrupa municípios em zonas geográficas usando clustering hierárquico Ward e nomeia as zonas pelos municípios de maior demanda.
545	`    return zonas`	Agrupa municípios em zonas geográficas usando clustering hierárquico Ward e nomeia as zonas pelos municípios de maior demanda.
546	``	Linha em branco usada para separar blocos.
547	``	Linha em branco usada para separar blocos.

Linha	Código	Explicação
548	# ---------- OTIMIZAÇÃO ----------	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.
549	DIM_COLUNA = {"LOCAIS": "S_LOCAIS", "ITENS": "S_ITENS", "H_TOTAL": "S_H_TOTAL", "KM": "S_KM"}	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.
550	``	Linha em branco usada para separar blocos.
551	``	Linha em branco usada para separar blocos.
552	def alocar_milp(pessoas, dias_disp, dm, zonas_idx, nome=""):	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.

Linha	Código	Explicação
553	"""Distribui as saídas de cada município entre os fiscais (atende 100% da demanda)"""	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.
554	P, M = len(pessoas), len(dm)	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.
555	Z = int(zonas_idx.max()) + 1	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.
556	T = dm["SAIDAS"].to_numpy(dtype=float)	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.

Linha	Código	Explicação
557	`    dims = list(PESOS)`	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.
558	`    contrib = {d: dm[DIM_COLUNA[d]].to_numpy() for d in dims}`	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.
559	`    A = np.asarray(dias_disp, dtype=float)`	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.
560	`    frac = A / A.sum()`	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.

Linha	Código	Explicação
561	`    alvo = {d: (contrib[d] * T).sum() * frac for d in dims}`	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.
562	```	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.
563	`    nx, ny, nz, nu = P * M, P * M, P * Z, len(dims)`	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.
564	`    n = nx + ny + nz + nu`	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.

Linha	Código	Explicação
565	`    ix = lambda p, m: p * M + m`	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.
566	`    iy = lambda p, m: nx + p * M + m`	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.
567	`    iz = lambda p, z: nx + ny + p * Z + z`	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.
568	`    iu = lambda k: nx + ny + nz + k`	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.

Linha	Código	Explicação
569	``	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.
570	lin, col, val, lb, ub = [], [], [], [], []	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.
571	r = 0	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.
572	``	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.

Linha	Código	Explicação
573	`    def add(entries, lo, hi):`	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.
574	`        nonlocal r`	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.
575	`        for c, v in entries:`	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.
576	`            lin.append(r); col.append(c); val.append(v)`	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.

Linha	Código	Explicação
577	`        lb.append(lo); ub.append(hi); r += 1`	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.
578	``	Linha em branco usada para separar blocos.
579	`    for m in range(M):`	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.
580	`        add([(ix(p, m), 1.0) for p in range(P)], T[m], T[m])`	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.
581	`        add([(iy(p, m), 1.0) for p in range(P)], 1, MAX_FISCAIS_POR_MUN)`	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.

Linha	Código	Explicação
582	`    for p in range(P):`	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.
583	`        for m in range(M):`	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.
584	`            minb = min(MIN_SAIDAS_BLOCO, T[m])`	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.
585	`            add([(ix(p, m), 1.0), (iy(p, m), -T[m])], -np.inf, 0)`	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.

Linha	Código	Explicação
586	add([(ix(p, m), 1.0), (iy(p, m), -minb)], 0, np.inf)	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.
587	add([(iy(p, m), 1.0), (iz(p, int(zonas_idx[m])), -1.0)], -np.inf, 0)	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.
588	add([(iy(p, m), 1.0) for m in range(M)], MIN_MUN_POR_FISCAL, MAX_MUN_POR_FISCAL)	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.
589	add([(ix(p, m), 1.0) for m in range(M)], 0, A[p] * USO_MAX_DIAS)	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.

Linha	Código	Explicação
590	`    for k, d in enumerate(dims):`	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.
591	`        c, t = contrib[d], alvo[d][p]`	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.
592	`        ent = [(ix(p, m), c[m]) for m in range(M)]`	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.
593	`        add(ent + [(iu(k), -t)], -np.inf, t)`	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.

Linha	Código	Explicação
594	add([(i, -v) for i, v in ent] + [(iu(k), -t)], -np.inf, -t)	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.
595	``	Linha em branco usada para separar blocos.
596	A_mat = coo_matrix((val, (lin, col)), shape=(r, n)).tocsr()	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.
597	c_obj = np.zeros(n)	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.
598	for k, d in enumerate(dims):	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.

Linha	Código	Explicação
599	`    c_obj[iu(k)] = PESOS[d]`	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.
600	`  c_obj[nx:nx + ny] = EPS_PAR`	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.
601	`  c_obj[nx + ny:nx + ny + nz] = EPS_ZONA`	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.
602	`  integrality = np.concatenate([np.ones(nx), np.ones(ny), np.ones(nz), np.zeros(nu)])`	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.

Linha	Código	Explicação
603	`    hi = np.concatenate([np.tile(T, P), np.ones(ny + nz), np.full(nu, np.inf)])`	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.
604	`    res = milp(c_obj, constraints=LinearConstraint(A_mat, np.array(lb), np.array(ub)), integrality=integrality,`	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.
605	`            bounds=Bounds(np.zeros(n), hi), options={"time_limit": TEMPO_LIMITE_S, "mip_rel_gap": 1e-3, "disp": False})`	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.
606	`    if res.x is None:`	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.

Linha	Código	Explicação
607	raise RuntimeError(f"Otimizador sem solução ({nome}): {res.message}")	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.
608	x = np.rint(res.x[:nx]).reshape(P, M).astype(int)	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.
609	info = {"metodo": "Otimização (MILP / HiGHS)", "status": int(res.status), "mensagem": str(res.message),	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.
610	"desvio_max": {d: float(res.x[iu(k)]) for k, d in enumerate(dims)}}	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.

Linha	Código	Explicação
611	`    return x, info`	Monta e resolve o modelo MILP: distribui todas as saídas entre fiscais, respeita limites por município, zonas, municípios por fiscal, dias disponíveis e minimiza desvios de locais, itens, horas e quilômetros.
612	``	Linha em branco usada para separar blocos.
613	``	Linha em branco usada para separar blocos.
614	`def alocar_guloso(pessoas, dias_disp, dm):`	Implementa o fallback guloso, alocando saídas por ordem de carga e sempre escolhendo o fiscal com menor carga ajustada pela disponibilidade.
615	`    P, M = len(pessoas), len(dm)`	Implementa o fallback guloso, alocando saídas por ordem de carga e sempre escolhendo o fiscal com menor carga ajustada pela disponibilidade.
616	`    A = np.asarray(dias_disp, dtype=float)`	Implementa o fallback guloso, alocando saídas por ordem de carga e sempre escolhendo o fiscal com menor carga ajustada pela disponibilidade.

Linha	Código	Explicação
617	`    frac = A / A.sum()`	Implementa o fallback guloso, alocando saídas por ordem de carga e sempre escolhendo o fiscal com menor carga ajustada pela disponibilidade.
618	`    T = dm["SAIDAS"].to_numpy()`	Implementa o fallback guloso, alocando saídas por ordem de carga e sempre escolhendo o fiscal com menor carga ajustada pela disponibilidade.
619	`    h = dm["S_H_TOTAL"].to_numpy()`	Implementa o fallback guloso, alocando saídas por ordem de carga e sempre escolhendo o fiscal com menor carga ajustada pela disponibilidade.
620	`    x = np.zeros((P, M), dtype=int)`	Implementa o fallback guloso, alocando saídas por ordem de carga e sempre escolhendo o fiscal com menor carga ajustada pela disponibilidade.
621	`    carga = np.zeros(P)`	Implementa o fallback guloso, alocando saídas por ordem de carga e sempre escolhendo o fiscal com menor carga ajustada pela disponibilidade.
622	`    for m in np.argsort(-h):`	Implementa o fallback guloso, alocando saídas por ordem de carga e sempre escolhendo o fiscal com menor carga ajustada pela disponibilidade.

Linha	Código	Explicação
623	`        for _ in range(int(T[m])):`	Implementa o fallback guloso, alocando saídas por ordem de carga e sempre escolhendo o fiscal com menor carga ajustada pela disponibilidade.
624	`            p = int(np.argmin(carga / frac))`	Implementa o fallback guloso, alocando saídas por ordem de carga e sempre escolhendo o fiscal com menor carga ajustada pela disponibilidade.
625	`            x[p, m] += 1`	Implementa o fallback guloso, alocando saídas por ordem de carga e sempre escolhendo o fiscal com menor carga ajustada pela disponibilidade.
626	`            carga[p] += h[m]`	Implementa o fallback guloso, alocando saídas por ordem de carga e sempre escolhendo o fiscal com menor carga ajustada pela disponibilidade.
627	`    return x`	Implementa o fallback guloso, alocando saídas por ordem de carga e sempre escolhendo o fiscal com menor carga ajustada pela disponibilidade.
628	``	Linha em branco usada para separar blocos.
629	``	Linha em branco usada para separar blocos.

Linha	Código	Explicação
630	def resolver(pessoas, dias_disp, dm, zonas_idx, nome):	Tenta MILP e, em caso de erro, registra o aviso e usa a solução gulosa.
631	try:	Tenta MILP e, em caso de erro, registra o aviso e usa a solução gulosa.
632	return alocar_milp(pessoas, dias_disp, dm, zonas_idx, nome)	Tenta MILP e, em caso de erro, registra o aviso e usa a solução gulosa.
633	except Exception as e:	Tenta MILP e, em caso de erro, registra o aviso e usa a solução gulosa.
634	print(f"AVISO ({nome}): otimização falhou ({type(e).__name__}: {e}); usando alocação gulosa.")	Tenta MILP e, em caso de erro, registra o aviso e usa a solução gulosa.
635	return alocar_guloso(pessoas, dias_disp, dm), {"metodo": "Gulosa (reserva)", "status": -1, "mensagem": str(e), "desvio_max": {}}	Tenta MILP e, em caso de erro, registra o aviso e usa a solução gulosa.
636	``	Linha em branco usada para separar blocos.
637	``	Linha em branco usada para separar blocos.
638	# ---------- DIVISÃO DOS LOCAIS E ITENS ENTRE OS FISCAIS ----------	Reparte estabelecimentos por município entre fiscais responsáveis, usando faixas geográficas contíguas e pesos de visitas, itens e horas; compara fiscal histórico e planejado.

Linha	Código	Explicação
639	def atribuir_locais(x, pessoas, dm, vis, df) -> pd.DataFrame:	Reparte estabelecimentos por município entre fiscais responsáveis, usando faixas geográficas contíguas e pesos de visitas, itens e horas; compara fiscal histórico e planejado.
640	"""Dentro de cada município, reparte os estabelecimentos entre os fiscais responsáveis, em faixas	Reparte estabelecimentos por município entre fiscais responsáveis, usando faixas geográficas contíguas e pesos de visitas, itens e horas; compara fiscal histórico e planejado.
641	geográficas contíguas (eixo principal das coordenadas), proporcionais às saídas de cada um e	Reparte estabelecimentos por município entre fiscais responsáveis, usando faixas geográficas contíguas e pesos de visitas, itens e horas; compara fiscal histórico e planejado.
642	equilibrando visitas, itens e horas."""	Reparte estabelecimentos por município entre fiscais responsáveis, usando faixas geográficas contíguas e pesos de visitas, itens e horas; compara fiscal histórico e planejado.

Linha	Código	Explicação
643	`    est = (vis.groupby("COD_PROPR").agg(NOMEFANTASIA=("NOMEFANTASIA", "first"), ENDERECO=("ENDERECO", "first"),`	Reparte estabelecimentos por município entre fiscais responsáveis, usando faixas geográficas contíguas e pesos de visitas, itens e horas; compara fiscal histórico e planejado.
644	`                                MUNICIPIO=("MUNICIPIO", "first"), BAIRRO=("BAIRRO", "first"),`	Reparte estabelecimentos por município entre fiscais responsáveis, usando faixas geográficas contíguas e pesos de visitas, itens e horas; compara fiscal histórico e planejado.
645	`                                LATITUDE=("LATITUDE", "first"), LONGITUDE=("LONGITUDE", "first"),`	Reparte estabelecimentos por município entre fiscais responsáveis, usando faixas geográficas contíguas e pesos de visitas, itens e horas; compara fiscal histórico e planejado.
646	`                                PRECISAO_GEO=("PRECISAO_GEO", "first"), VISITAS=("DIA", "size"), ITENS=("ITENS", "sum"),`	Reparte estabelecimentos por município entre fiscais responsáveis, usando faixas geográficas contíguas e pesos de visitas, itens e horas; compara fiscal histórico e planejado.

Linha	Código	Explicação
647	H_FISC=("H_FISC", "sum")).reset_index())	Reparte estabelecimentos por município entre fiscais responsáveis, usando faixas geográficas contíguas e pesos de visitas, itens e horas; compara fiscal histórico e planejado.
648	hist = vis.groupby(["COD_PROPR", "FISCAL"]).size().rename("N").reset_index().sort_values("N", ascending=False)	Reparte estabelecimentos por município entre fiscais responsáveis, usando faixas geográficas contíguas e pesos de visitas, itens e horas; compara fiscal histórico e planejado.
649	est["FISCAL_HISTORICO"] = est["COD_PROPR"].map(hist.drop_duplicates("COD_PROPR").set_index("COD_PROPR")["FISCAL"])	Reparte estabelecimentos por município entre fiscais responsáveis, usando faixas geográficas contíguas e pesos de visitas, itens e horas; compara fiscal histórico e planejado.
650	est["FISCAL_PLANO"] = None	Reparte estabelecimentos por município entre fiscais responsáveis, usando faixas geográficas contíguas e pesos de visitas, itens e horas; compara fiscal histórico e planejado.

Linha	Código	Explicação
651	`    pessoas = list(pessoas)`	Reparte estabelecimentos por município entre fiscais responsáveis, usando faixas geográficas contíguas e pesos de visitas, itens e horas; compara fiscal histórico e planejado.
652	`    for m, mun in enumerate(dm["MUNICIPIO"]):`	Reparte estabelecimentos por município entre fiscais responsáveis, usando faixas geográficas contíguas e pesos de visitas, itens e horas; compara fiscal histórico e planejado.
653	`        resp = [(p, x[p, m]) for p in range(len(pessoas)) if x[p, m] > 0]`	Reparte estabelecimentos por município entre fiscais responsáveis, usando faixas geográficas contíguas e pesos de visitas, itens e horas; compara fiscal histórico e planejado.
654	`        idx = est.index[est["MUNICIPIO"] == mun]`	Reparte estabelecimentos por município entre fiscais responsáveis, usando faixas geográficas contíguas e pesos de visitas, itens e horas; compara fiscal histórico e planejado.

Linha	Código	Explicação
655	`    if not len(idx):`	Reparte estabelecimentos por município entre fiscais responsáveis, usando faixas geográficas contíguas e pesos de visitas, itens e horas; compara fiscal histórico e planejado.
656	`        continue`	Reparte estabelecimentos por município entre fiscais responsáveis, usando faixas geográficas contíguas e pesos de visitas, itens e horas; compara fiscal histórico e planejado.
657	`    e = est.loc[idx].copy()`	Reparte estabelecimentos por município entre fiscais responsáveis, usando faixas geográficas contíguas e pesos de visitas, itens e horas; compara fiscal histórico e planejado.
658	`    if len(resp) == 1:`	Reparte estabelecimentos por município entre fiscais responsáveis, usando faixas geográficas contíguas e pesos de visitas, itens e horas; compara fiscal histórico e planejado.

Linha	Código	Explicação
659	`            est.loc[idx, "FISCAL_PLANO"] = pessoas[resp[0][0]]`	Reparte estabelecimentos por município entre fiscais responsáveis, usando faixas geográficas contíguas e pesos de visitas, itens e horas; compara fiscal histórico e planejado.
660	`            continue`	Reparte estabelecimentos por município entre fiscais responsáveis, usando faixas geográficas contíguas e pesos de visitas, itens e horas; compara fiscal histórico e planejado.
661	`        xy = np.column_stack([e["LONGITUDE"].fillna(e["LONGITUDE"].mean()), e["LATITUDE"].fillna(e["LATITUDE"].mean())])`	Reparte estabelecimentos por município entre fiscais responsáveis, usando faixas geográficas contíguas e pesos de visitas, itens e horas; compara fiscal histórico e planejado.
662	`        xy = xy - xy.mean(axis=0)`	Reparte estabelecimentos por município entre fiscais responsáveis, usando faixas geográficas contíguas e pesos de visitas, itens e horas; compara fiscal histórico e planejado.

Linha	Código	Explicação
663	`        eixo = np.linalg.svd(xy, full_matrices=False)[2][0] if len(e) > 1 else np.array([1.0, 0.0])`	Reparte estabelecimentos por município entre fiscais responsáveis, usando faixas geográficas contíguas e pesos de visitas, itens e horas; compara fiscal histórico e planejado.
664	`        e["ORDEM"] = xy @ eixo`	Reparte estabelecimentos por município entre fiscais responsáveis, usando faixas geográficas contíguas e pesos de visitas, itens e horas; compara fiscal histórico e planejado.
665	`        e = e.sort_values("ORDEM")`	Reparte estabelecimentos por município entre fiscais responsáveis, usando faixas geográficas contíguas e pesos de visitas, itens e horas; compara fiscal histórico e planejado.
666	`        peso = (e["VISITAS"] / e["VISITAS"].sum() + e["ITENS"] / max(e["ITENS"].sum(), 1) + e["H_FISC"] / max(e["H_FISC"].sum(), 1e-9)) / 3.0`	Reparte estabelecimentos por município entre fiscais responsáveis, usando faixas geográficas contíguas e pesos de visitas, itens e horas; compara fiscal histórico e planejado.

Linha	Código	Explicação
667	`    cum = (peso.cumsum() - peso / 2).to_numpy()                    # ponto médio de cada local no intervalo acumulado`	Reparte estabelecimentos por município entre fiscais responsáveis, usando faixas geográficas contíguas e pesos de visitas, itens e horas; compara fiscal histórico e planejado.
668	`    compartilhar = np.array([s for _, s in resp], dtype=float)`	Reparte estabelecimentos por município entre fiscais responsáveis, usando faixas geográficas contíguas e pesos de visitas, itens e horas; compara fiscal histórico e planejado.
669	`    limites = np.cumsum(compartilhar / compartilhar.sum())`	Reparte estabelecimentos por município entre fiscais responsáveis, usando faixas geográficas contíguas e pesos de visitas, itens e horas; compara fiscal histórico e planejado.
670	`    dono = np.searchsorted(limites, cum, side="right").clip(max=len(resp) - 1)`	Reparte estabelecimentos por município entre fiscais responsáveis, usando faixas geográficas contíguas e pesos de visitas, itens e horas; compara fiscal histórico e planejado.

Linha	Código	Explicação
671	`    est.loc[e.index, "FISCAL_PLANO"] = [pessoas[resp[i][0]] for i in dono]`	Reparte estabelecimentos por município entre fiscais responsáveis, usando faixas geográficas contíguas e pesos de visitas, itens e horas; compara fiscal histórico e planejado.
672	`  est["MUDOU_DE_FISCAL"] = est["FISCAL_PLANO"] != est["FISCAL_HISTORICO"]`	Reparte estabelecimentos por município entre fiscais responsáveis, usando faixas geográficas contíguas e pesos de visitas, itens e horas; compara fiscal histórico e planejado.
673	`  return est`	Reparte estabelecimentos por município entre fiscais responsáveis, usando faixas geográficas contíguas e pesos de visitas, itens e horas; compara fiscal histórico e planejado.
674	``	Linha em branco usada para separar blocos.
675	``	Linha em branco usada para separar blocos.
676	`def corrigir_saidas_sem_locais(x, pessoas, dm, est):`	Corrige alocações em que um fiscal recebeu saídas mas nenhum local, transferindo-as para um responsável que possui locais no município.

Linha	Código	Explicação
677	"""Se um fiscal ficou com saídas em um município onde não recebeu nenhum local, transfere essas saídas a quem recebeu."""	Corrige alocações em que um fiscal recebeu saídas mas nenhum local, transferindo-as para um responsável que possui locais no município.
678	pessoas = list(pessoas)	Corrige alocações em que um fiscal recebeu saídas mas nenhum local, transferindo-as para um responsável que possui locais no município.
679	mudou = 0	Corrige alocações em que um fiscal recebeu saídas mas nenhum local, transferindo-as para um responsável que possui locais no município.
680	for m, mun in enumerate(dm["MUNICIPIO"]):	Corrige alocações em que um fiscal recebeu saídas mas nenhum local, transferindo-as para um responsável que possui locais no município.
681	com_local = set(est.loc[est["MUNICIPIO"] == mun, "FISCAL_PLANO"].dropna())	Corrige alocações em que um fiscal recebeu saídas mas nenhum local, transferindo-as para um responsável que possui locais no município.
682	if not com_local:	Corrige alocações em que um fiscal recebeu saídas mas nenhum local, transferindo-as para um responsável que possui locais no município.

Linha	Código	Explicação
683	continue	Corrige alocações em que um fiscal recebeu saídas mas nenhum local, transferindo-as para um responsável que possui locais no município.
684	for p in range(len(pessoas)):	Corrige alocações em que um fiscal recebeu saídas mas nenhum local, transferindo-as para um responsável que possui locais no município.
685	if x[p, m] > 0 and pessoas[p] not in com_local:	Corrige alocações em que um fiscal recebeu saídas mas nenhum local, transferindo-as para um responsável que possui locais no município.
686	alvo = pessoas.index(max(com_local, key=lambda f: x[pessoas.index(f), m]))	Corrige alocações em que um fiscal recebeu saídas mas nenhum local, transferindo-as para um responsável que possui locais no município.
687	x[alvo, m] += x[p, m]	Corrige alocações em que um fiscal recebeu saídas mas nenhum local, transferindo-as para um responsável que possui locais no município.
688	x[p, m] = 0	Corrige alocações em que um fiscal recebeu saídas mas nenhum local, transferindo-as para um responsável que possui locais no município.

Linha	Código	Explicação
689	mudou += 1	Corrige alocações em que um fiscal recebeu saídas mas nenhum local, transferindo-as para um responsável que possui locais no município.
690	return x, mudou	Corrige alocações em que um fiscal recebeu saídas mas nenhum local, transferindo-as para um responsável que possui locais no município.
691	``	Linha em branco usada para separar blocos.
692	``	Linha em branco usada para separar blocos.
693	# ---------- TABELAS DO PLANO ----------	Cria a tabela longa do plano por fiscal e município, com saídas, locais, itens, horas, quilômetros e percentual de responsabilidade.
694	def plano_longo(x, pessoas, dm, est, vis) -> pd.DataFrame:	Cria a tabela longa do plano por fiscal e município, com saídas, locais, itens, horas, quilômetros e percentual de responsabilidade.
695	pessoas = list(pessoas)	Cria a tabela longa do plano por fiscal e município, com saídas, locais, itens, horas, quilômetros e percentual de responsabilidade.

Linha	Código	Explicação
696	`    por = est.groupby(["MUNICIPIO", "FISCAL_PLANO"]).agg(LOCAIS=("VISITAS", "sum"), ESTABELECIMENTOS=("COD_PROPR", "nunique"),`	Cria a tabela longa do plano por fiscal e município, com saídas, locais, itens, horas, quilômetros e percentual de responsabilidade.
697	`                                                ITENS=("ITENS", "sum"), H_FISCALIZACAO=("H_FISC", "sum")).reset_index()`	Cria a tabela longa do plano por fiscal e município, com saídas, locais, itens, horas, quilômetros e percentual de responsabilidade.
698	`    linhas = []`	Cria a tabela longa do plano por fiscal e município, com saídas, locais, itens, horas, quilômetros e percentual de responsabilidade.
699	`    for _, r in por.iterrows():`	Cria a tabela longa do plano por fiscal e município, com saídas, locais, itens, horas, quilômetros e percentual de responsabilidade.
700	`        m = int(np.where(dm["MUNICIPIO"].values == r["MUNICIPIO"])[0][0])`	Cria a tabela longa do plano por fiscal e município, com saídas, locais, itens, horas, quilômetros e percentual de responsabilidade.
701	`        p = pessoas.index(r["FISCAL_PLANO"])`	Cria a tabela longa do plano por fiscal e município, com saídas, locais, itens, horas, quilômetros e percentual de responsabilidade.
702	`        s = int(x[p, m]); d = dm.iloc[m]`	Cria a tabela longa do plano por fiscal e município, com saídas, locais, itens, horas, quilômetros e percentual de responsabilidade.

Linha	Código	Explicação
703	`    linhas.append({"PESSOA": r["FISCAL_PLANO"], "MUNICIPIO": r["MUNICIPIO"], "SAIDAS": s, "LOCAIS": int(r["LOCAIS"]),`	Cria a tabela longa do plano por fiscal e município, com saídas, locais, itens, horas, quilômetros e percentual de responsabilidade.
704	`                "ESTABELECIMENTOS": int(r["ESTABELECIMENTOS"]), "ITENS": int(r["ITENS"]),`	Cria a tabela longa do plano por fiscal e município, com saídas, locais, itens, horas, quilômetros e percentual de responsabilidade.
705	`                "H_FISCALIZACAO": r["H_FISCALIZACAO"] * FATOR_DEMANDA, "H_DESLOC": s * d["S_H_DESLOC"],`	Cria a tabela longa do plano por fiscal e município, com saídas, locais, itens, horas, quilômetros e percentual de responsabilidade.
706	`                "H_TOTAL": r["H_FISCALIZACAO"] * FATOR_DEMANDA + s * d["S_H_DESLOC"], "KM": s * d["S_KM"],`	Cria a tabela longa do plano por fiscal e município, com saídas, locais, itens, horas, quilômetros e percentual de responsabilidade.
707	`                "PCT_DO_MUNICIPIO": s / d["SAIDAS"]})`	Cria a tabela longa do plano por fiscal e município, com saídas, locais, itens, horas, quilômetros e percentual de responsabilidade.
708	`    return pd.DataFrame(linhas)`	Cria a tabela longa do plano por fiscal e município, com saídas, locais, itens, horas, quilômetros e percentual de responsabilidade.
709	``	Linha em branco usada para separar blocos.
710	``	Linha em branco usada para separar blocos.

Linha	Código	Explicação
711	def plano_anual(longo: pd.DataFrame, pessoas, dias_disp) -> pd.DataFrame:	Agrega o plano anual por fiscal e calcula métricas normalizadas por dia disponível e por saída.
712	a = longo.groupby("PESSOA").agg(SAIDAS=("SAIDAS", "sum"), LOCAIS=("LOCAIS", "sum"), ESTABELECIMENTOS=("ESTABELECIMENTOS", "sum"),	Agrega o plano anual por fiscal e calcula métricas normalizadas por dia disponível e por saída.
713	ITENS=("ITENS", "sum"), H_FISCALIZACAO=("H_FISCALIZACAO", "sum"), H_DESLOC=("H_DESLOC", "sum"),	Agrega o plano anual por fiscal e calcula métricas normalizadas por dia disponível e por saída.
714	H_TOTAL=("H_TOTAL", "sum"), KM=("KM", "sum"), MUNICIPIOS=("MUNICIPIO", "nunique"))	Agrega o plano anual por fiscal e calcula métricas normalizadas por dia disponível e por saída.
715	a = a.reindex(pessoas).fillna(0)	Agrega o plano anual por fiscal e calcula métricas normalizadas por dia disponível e por saída.
716	for c in ("SAIDAS", "LOCAIS", "ESTABELECIMENTOS", "ITENS", "MUNICIPIOS"):	Agrega o plano anual por fiscal e calcula métricas normalizadas por dia disponível e por saída.
717	a[c] = a[c].astype(int)	Agrega o plano anual por fiscal e calcula métricas normalizadas por dia disponível e por saída.
718	a["DIAS_DISPONIVEIS"] = list(dias_disp)	Agrega o plano anual por fiscal e calcula métricas normalizadas por dia disponível e por saída.

Linha	Código	Explicação
719	`    dd = a["DIAS_DISPONIVEIS"].replace(0, np.nan)`	Agrega o plano anual por fiscal e calcula métricas normalizadas por dia disponível e por saída.
720	`    a["PCT_DIAS_COM_SAIDA"] = a["SAIDAS"] / dd`	Agrega o plano anual por fiscal e calcula métricas normalizadas por dia disponível e por saída.
721	`    a["H_TOTAL_POR_DIA_DISP"] = a["H_TOTAL"] / dd`	Agrega o plano anual por fiscal e calcula métricas normalizadas por dia disponível e por saída.
722	`    a["KM_POR_DIA_DISP"] = a["KM"] / dd`	Agrega o plano anual por fiscal e calcula métricas normalizadas por dia disponível e por saída.
723	`    a["H_POR_SAIDA"] = a["H_TOTAL"] / a["SAIDAS"].replace(0, np.nan)`	Agrega o plano anual por fiscal e calcula métricas normalizadas por dia disponível e por saída.
724	`    a["KM_POR_SAIDA"] = a["KM"] / a["SAIDAS"].replace(0, np.nan)`	Agrega o plano anual por fiscal e calcula métricas normalizadas por dia disponível e por saída.
725	`    a["LOCAIS_POR_SAIDA"] = a["LOCAIS"] / a["SAIDAS"].replace(0, np.nan)`	Agrega o plano anual por fiscal e calcula métricas normalizadas por dia disponível e por saída.
726	`    a["ITENS_POR_SAIDA"] = a["ITENS"] / a["SAIDAS"].replace(0, np.nan)`	Agrega o plano anual por fiscal e calcula métricas normalizadas por dia disponível e por saída.

Linha	Código	Explicação
727	__return a.reset_index().rename(columns={"index": "PESSOA"})	Agrega o plano anual por fiscal e calcula métricas normalizadas por dia disponível e por saída.
728	``	Linha em branco usada para separar blocos.
729	``	Linha em branco usada para separar blocos.
730	MEDIDAS = ["SAIDAS", "LOCAIS", "ITENS", "H_FISCALIZACAO", "H_DESLOC", "H_TOTAL", "KM"]	Define medidas, medidas inteiras e medidas que recebem faixa de tolerância.
731	INTEIRAS = ["SAIDAS", "LOCAIS", "ITENS"]	Define medidas, medidas inteiras e medidas que recebem faixa de tolerância.
732	COM_FAIXA = ["LOCAIS", "ITENS", "H_TOTAL", "KM"]	Define medidas, medidas inteiras e medidas que recebem faixa de tolerância.
733	``	Linha em branco usada para separar blocos.
734	``	Linha em branco usada para separar blocos.
735	def metas_periodicas(anual: pd.DataFrame, disp: pd.DataFrame, cal: pd.DataFrame, nivel: str, inteiro: bool) -> pd.DataFrame:	Converte metas anuais em metas mensais ou semanais proporcionalmente aos dias disponíveis e cria limites mínimo/máximo de 10%.

Linha	Código	Explicação
736	`    chave = cal.set_index("DATA")[nivel].reindex(disp.index)`	Converte metas anuais em metas mensais ou semanais proporcionalmente aos dias disponíveis e cria limites mínimo/máximo de 10%.
737	`    dias = disp.groupby(chave).sum()`	Converte metas anuais em metas mensais ou semanais proporcionalmente aos dias disponíveis e cria limites mínimo/máximo de 10%.
738	`    inicio = cal.groupby(nivel)["DATA"].min()`	Converte metas anuais em metas mensais ou semanais proporcionalmente aos dias disponíveis e cria limites mínimo/máximo de 10%.
739	`    fim = cal.groupby(nivel)["DATA"].max()`	Converte metas anuais em metas mensais ou semanais proporcionalmente aos dias disponíveis e cria limites mínimo/máximo de 10%.
740	`    linhas = []`	Converte metas anuais em metas mensais ou semanais proporcionalmente aos dias disponíveis e cria limites mínimo/máximo de 10%.

Linha	Código	Explicação
741	`    for _, a in anual.iterrows():`	Converte metas anuais em metas mensais ou semanais proporcionalmente aos dias disponíveis e cria limites mínimo/máximo de 10%.
742	`        w = dias[a["PESSOA"]].astype(float).to_numpy()`	Converte metas anuais em metas mensais ou semanais proporcionalmente aos dias disponíveis e cria limites mínimo/máximo de 10%.
743	`        tot = w.sum()`	Converte metas anuais em metas mensais ou semanais proporcionalmente aos dias disponíveis e cria limites mínimo/máximo de 10%.
744	`        vals = {}`	Converte metas anuais em metas mensais ou semanais proporcionalmente aos dias disponíveis e cria limites mínimo/máximo de 10%.
745	`        for med in MEDIDAS:`	Converte metas anuais em metas mensais ou semanais proporcionalmente aos dias disponíveis e cria limites mínimo/máximo de 10%.

Linha	Código	Explicação
746	`        if inteiro and med in INTEIRAS:`	Converte metas anuais em metas mensais ou semanais proporcionalmente aos dias disponíveis e cria limites mínimo/máximo de 10%.
747	`            vals[med] = distribuir_inteiro(a[med], w)`	Converte metas anuais em metas mensais ou semanais proporcionalmente aos dias disponíveis e cria limites mínimo/máximo de 10%.
748	`        else:`	Converte metas anuais em metas mensais ou semanais proporcionalmente aos dias disponíveis e cria limites mínimo/máximo de 10%.
749	`            vals[med] = a[med] * w / tot if tot else w * 0`	Converte metas anuais em metas mensais ou semanais proporcionalmente aos dias disponíveis e cria limites mínimo/máximo de 10%.
750	`    for i, per in enumerate(dias.index):`	Converte metas anuais em metas mensais ou semanais proporcionalmente aos dias disponíveis e cria limites mínimo/máximo de 10%.

Linha	Código	Explicação
751	row = {"PESSOA": a["PESSOA"], nivel: int(per), "INICIO": inicio[per].date(), "FIM": fim[per].date(),	Converte metas anuais em metas mensais ou semanais proporcionalmente aos dias disponíveis e cria limites mínimo/máximo de 10%.
752	"DIAS_UTEIS_DISPONIVEIS": int(w[i])}	Converte metas anuais em metas mensais ou semanais proporcionalmente aos dias disponíveis e cria limites mínimo/máximo de 10%.
753	for med in MEDIDAS:	Converte metas anuais em metas mensais ou semanais proporcionalmente aos dias disponíveis e cria limites mínimo/máximo de 10%.
754	row[med] = vals[med][i]	Converte metas anuais em metas mensais ou semanais proporcionalmente aos dias disponíveis e cria limites mínimo/máximo de 10%.
755	for med in COM_FAIXA:	Converte metas anuais em metas mensais ou semanais proporcionalmente aos dias disponíveis e cria limites mínimo/máximo de 10%.

Linha	Código	Explicação
756	row[f"{med}_MIN"] = row[med] * (1 - TOLERANCIA_META)	Converte metas anuais em metas mensais ou semanais proporcionalmente aos dias disponíveis e cria limites mínimo/máximo de 10%.
757	row[f"{med}_MAX"] = row[med] * (1 + TOLERANCIA_META)	Converte metas anuais em metas mensais ou semanais proporcionalmente aos dias disponíveis e cria limites mínimo/máximo de 10%.
758	linhas.append(row)	Converte metas anuais em metas mensais ou semanais proporcionalmente aos dias disponíveis e cria limites mínimo/máximo de 10%.
759	return pd.DataFrame(linhas)	Converte metas anuais em metas mensais ou semanais proporcionalmente aos dias disponíveis e cria limites mínimo/máximo de 10%.
760	``	Linha em branco usada para separar blocos.
761	``	Linha em branco usada para separar blocos.
762	# ---------- HISTÓRICO E INDICADORES ----------	Cria grades completas de períodos, incluindo períodos sem atividade com valores zero.

Linha	Código	Explicação
763	def grade_periodos(obs: pd.DataFrame, pessoas: list, cal: pd.DataFrame, nivel: str) -> pd.DataFrame:	Cria grades completas de períodos, incluindo períodos sem atividade com valores zero.
764	o = obs.copy()	Cria grades completas de períodos, incluindo períodos sem atividade com valores zero.
765	o[nivel] = o["DIA"].map(cal.set_index("DATA")[nivel])	Cria grades completas de períodos, incluindo períodos sem atividade com valores zero.
766	o = o.dropna(subset=[nivel])	Cria grades completas de períodos, incluindo períodos sem atividade com valores zero.
767	o[nivel] = o[nivel].astype(int)	Cria grades completas de períodos, incluindo períodos sem atividade com valores zero.
768	g = o.groupby(["PESSOA", nivel])[MEDIDAS].sum()	Cria grades completas de períodos, incluindo períodos sem atividade com valores zero.
769	idx = pd.MultiIndex.from_product([pessoas, sorted(cal[nivel].unique())], names=["PESSOA", nivel])	Cria grades completas de períodos, incluindo períodos sem atividade com valores zero.
770	return g.reindex(idx, fill_value=0).reset_index()	Cria grades completas de períodos, incluindo períodos sem atividade com valores zero.

Linha	Código	Explicação
771	``	Linha em branco usada para separar blocos.
772	``	Linha em branco usada para separar blocos.
773	def indicadores_pessoas(obs, pessoas, disp, cal) -> pd.DataFrame:	Calcula indicadores por fiscal: dias, locais, itens, horas, km, médias, CV, dias acima da jornada e meses sem atividade.
774	gm = grade_periodos(obs, pessoas, cal, "MES")	Calcula indicadores por fiscal: dias, locais, itens, horas, km, médias, CV, dias acima da jornada e meses sem atividade.
775	gs = grade_periodos(obs, pessoas, cal, "SEMANA")	Calcula indicadores por fiscal: dias, locais, itens, horas, km, médias, CV, dias acima da jornada e meses sem atividade.
776	mes_k = cal.set_index("DATA")["MES"].reindex(disp.index)	Calcula indicadores por fiscal: dias, locais, itens, horas, km, médias, CV, dias acima da jornada e meses sem atividade.
777	sem_k = cal.set_index("DATA")["SEMANA"].reindex(disp.index)	Calcula indicadores por fiscal: dias, locais, itens, horas, km, médias, CV, dias acima da jornada e meses sem atividade.
778	linhas = []	Calcula indicadores por fiscal: dias, locais, itens, horas, km, médias, CV, dias acima da jornada e meses sem atividade.

Linha	Código	Explicação
779	`    for p in pessoas:`	Calcula indicadores por fiscal: dias, locais, itens, horas, km, médias, CV, dias acima da jornada e meses sem atividade.
780	`        o = obs[obs["PESSOA"] == p]`	Calcula indicadores por fiscal: dias, locais, itens, horas, km, médias, CV, dias acima da jornada e meses sem atividade.
781	`        dm_mes = disp[p].groupby(mes_k).sum()`	Calcula indicadores por fiscal: dias, locais, itens, horas, km, médias, CV, dias acima da jornada e meses sem atividade.
782	`        dm_sem = disp[p].groupby(sem_k).sum()`	Calcula indicadores por fiscal: dias, locais, itens, horas, km, médias, CV, dias acima da jornada e meses sem atividade.
783	`        m = gm[gm["PESSOA"] == p]; s = gs[gs["PESSOA"] == p]`	Calcula indicadores por fiscal: dias, locais, itens, horas, km, médias, CV, dias acima da jornada e meses sem atividade.
784	`        m_ok, s_ok = m[m["MES"].map(dm_mes) > 0], s[s["SEMANA"].map(dm_sem) > 0]`	Calcula indicadores por fiscal: dias, locais, itens, horas, km, médias, CV, dias acima da jornada e meses sem atividade.
785	`        linhas.append({"FISCAL": p, "DIAS_DISPONIVEIS": int(disp[p].sum()), "DIAS_TRABALHADOS": len(o),`	Calcula indicadores por fiscal: dias, locais, itens, horas, km, médias, CV, dias acima da jornada e meses sem atividade.

Linha	Código	Explicação
786	"LOCAIS": o["LOCAIS"].sum(), "ITENS": o["ITENS"].sum(), "H_TOTAL": o["H_TOTAL"].sum(), "KM": o["KM"].sum(),	Calcula indicadores por fiscal: dias, locais, itens, horas, km, médias, CV, dias acima da jornada e meses sem atividade.
787	"H_MEDIA_POR_DIA_TRAB": o["H_TOTAL"].mean() if len(o) else np.nan,	Calcula indicadores por fiscal: dias, locais, itens, horas, km, médias, CV, dias acima da jornada e meses sem atividade.
788	"KM_MEDIO_POR_DIA_TRAB": o["KM"].mean() if len(o) else np.nan,	Calcula indicadores por fiscal: dias, locais, itens, horas, km, médias, CV, dias acima da jornada e meses sem atividade.
789	"LOCAIS_MEDIOS_POR_DIA_TRAB": o["LOCAIS"].mean() if len(o) else np.nan,	Calcula indicadores por fiscal: dias, locais, itens, horas, km, médias, CV, dias acima da jornada e meses sem atividade.
790	"ITENS_MEDIOS_POR_DIA_TRAB": o["ITENS"].mean() if len(o) else np.nan,	Calcula indicadores por fiscal: dias, locais, itens, horas, km, médias, CV, dias acima da jornada e meses sem atividade.
791	"CV_DIARIO_H": cv(o["H_TOTAL"]) if len(o) else np.nan,	Calcula indicadores por fiscal: dias, locais, itens, horas, km, médias, CV, dias acima da jornada e meses sem atividade.
792	"PCT_DIAS_ACIMA_JORNADA": float((o["H_TOTAL"] > JORNADA_MAX_H).mean()) if len(o) else np.nan,	Calcula indicadores por fiscal: dias, locais, itens, horas, km, médias, CV, dias acima da jornada e meses sem atividade.

Linha	Código	Explicação
793	"CV_SEMANAL_H": cv(s_ok["H_TOTAL"]), "CV_MENSAL_H": cv(m_ok["H_TOTAL"]),	Calcula indicadores por fiscal: dias, locais, itens, horas, km, médias, CV, dias acima da jornada e meses sem atividade.
794	"CV_MENSAL_KM": cv(m_ok["KM"]), "CV_MENSAL_ITENS": cv(m_ok["ITENS"]), "CV_MENSAL_LOCAIS": cv(m_ok["LOCAIS"]),	Calcula indicadores por fiscal: dias, locais, itens, horas, km, médias, CV, dias acima da jornada e meses sem atividade.
795	"MESES_SEM_ATIVIDADE": int((m_ok["H_TOTAL"] == 0).sum()),	Calcula indicadores por fiscal: dias, locais, itens, horas, km, médias, CV, dias acima da jornada e meses sem atividade.
796	"MES_MAIOR_CARGA_H": m["H_TOTAL"].max(), "MES_MENOR_CARGA_H": m_ok["H_TOTAL"].min() if len(m_ok) else np.nan})	Calcula indicadores por fiscal: dias, locais, itens, horas, km, médias, CV, dias acima da jornada e meses sem atividade.
797	return pd.DataFrame(linhas)	Calcula indicadores por fiscal: dias, locais, itens, horas, km, médias, CV, dias acima da jornada e meses sem atividade.
798	``	Linha em branco usada para separar blocos.
799	``	Linha em branco usada para separar blocos.
800	MEDIDAS_IND = {"LOCAIS": "Locais fiscalizados", "ITENS": "Itens fiscalizados", "H_TOTAL": "Horas trabalhadas", "KM": "Quilômetros"}	Calcula indicadores de equilíbrio do grupo entre fiscais e regularidade do calendário nos níveis anual, mensal e semanal.

Linha	Código	Explicação
801	``	Linha em branco usada para separar blocos.
802	``	Linha em branco usada para separar blocos.
803	def indicadores_grupo(grade_mes, grade_sem, disp, cal, pessoas) -> list:	Calcula indicadores de equilíbrio do grupo entre fiscais e regularidade do calendário nos níveis anual, mensal e semanal.
804	__mes_k = cal.set_index("DATA")["MES"].reindex(disp.index)	Calcula indicadores de equilíbrio do grupo entre fiscais e regularidade do calendário nos níveis anual, mensal e semanal.
805	__sem_k = cal.set_index("DATA")["SEMANA"].reindex(disp.index)	Calcula indicadores de equilíbrio do grupo entre fiscais e regularidade do calendário nos níveis anual, mensal e semanal.
806	__saida = []	Calcula indicadores de equilíbrio do grupo entre fiscais e regularidade do calendário nos níveis anual, mensal e semanal.
807	__for med, nome in MEDIDAS_IND.items():	Calcula indicadores de equilíbrio do grupo entre fiscais e regularidade do calendário nos níveis anual, mensal e semanal.
808	____anual = grade_mes.groupby("PESSOA")[med].sum().reindex(pessoas)	Calcula indicadores de equilíbrio do grupo entre fiscais e regularidade do calendário nos níveis anual, mensal e semanal.

Linha	Código	Explicação
809	`        dias_ano = disp.sum().reindex(pessoas)`	Calcula indicadores de equilíbrio do grupo entre fiscais e regularidade do calendário nos níveis anual, mensal e semanal.
810	`        saida.append(("Anual", nome, "Entre fiscais (CV da carga por dia disponível)", cv((anual / dias_ano.replace(0, np.nan)).dropna())))`	Calcula indicadores de equilíbrio do grupo entre fiscais e regularidade do calendário nos níveis anual, mensal e semanal.
811	`        for nivel, g, k in (("Mensal", grade_mes, mes_k), ("Semanal", grade_sem, sem_k)):`	Calcula indicadores de equilíbrio do grupo entre fiscais e regularidade do calendário nos níveis anual, mensal e semanal.
812	`            col = "MES" if nivel == "Mensal" else "SEMANA"`	Calcula indicadores de equilíbrio do grupo entre fiscais e regularidade do calendário nos níveis anual, mensal e semanal.
813	`            dias = disp.groupby(k).sum()`	Calcula indicadores de equilíbrio do grupo entre fiscais e regularidade do calendário nos níveis anual, mensal e semanal.
814	`            cvs, eq, eqd = [], [], []`	Calcula indicadores de equilíbrio do grupo entre fiscais e regularidade do calendário nos níveis anual, mensal e semanal.
815	`            for per, gp in g.groupby(col):`	Calcula indicadores de equilíbrio do grupo entre fiscais e regularidade do calendário nos níveis anual, mensal e semanal.

Linha	Código	Explicação
816	`            d = dias.loc[per].reindex(pessoas)`	Calcula indicadores de equilíbrio do grupo entre fiscais e regularidade do calendário nos níveis anual, mensal e semanal.
817	`            v = gp.set_index("PESSOA")[med].reindex(pessoas)`	Calcula indicadores de equilíbrio do grupo entre fiscais e regularidade do calendário nos níveis anual, mensal e semanal.
818	`            ok = d > 0`	Calcula indicadores de equilíbrio do grupo entre fiscais e regularidade do calendário nos níveis anual, mensal e semanal.
819	`            if ok.sum() >= 2:`	Calcula indicadores de equilíbrio do grupo entre fiscais e regularidade do calendário nos níveis anual, mensal e semanal.
820	`                cvs.append(cv((v[ok] / d[ok]).values))`	Calcula indicadores de equilíbrio do grupo entre fiscais e regularidade do calendário nos níveis anual, mensal e semanal.
821	`            eq.append(v[ok].sum()); eqd.append(d[ok].sum())`	Calcula indicadores de equilíbrio do grupo entre fiscais e regularidade do calendário nos níveis anual, mensal e semanal.
822	`        saida.append((nivel, nome, "Entre fiscais (média dos períodos do CV da carga por dia disponível)", float(np.nanmean(cvs)) if cvs else np.nan))`	Calcula indicadores de equilíbrio do grupo entre fiscais e regularidade do calendário nos níveis anual, mensal e semanal.

Linha	Código	Explicação
823	serie = pd.Series(eq, dtype=float) / pd.Series(eqd, dtype=float).replace(0, np.nan)	Calcula indicadores de equilíbrio do grupo entre fiscais e regularidade do calendário nos níveis anual, mensal e semanal.
824	saida.append((nivel, nome, "Regularidade da equipe no calendário (CV entre períodos)", cv(serie.dropna())))	Calcula indicadores de equilíbrio do grupo entre fiscais e regularidade do calendário nos níveis anual, mensal e semanal.
825	return saida	Calcula indicadores de equilíbrio do grupo entre fiscais e regularidade do calendário nos níveis anual, mensal e semanal.
826	``	Linha em branco usada para separar blocos.
827	``	Linha em branco usada para separar blocos.
828	def indicador_diario(hist_dia: pd.DataFrame, dm: pd.DataFrame) -> list:	Compara variabilidade diária atual e planejada e percentual de saídas acima da jornada.
829	pesos = dm["SAIDAS"].to_numpy()	Compara variabilidade diária atual e planejada e percentual de saídas acima da jornada.
830	plano = {"LOCAIS": np.repeat(dm["S_LOCAIS"].to_numpy(), pesos), "ITENS": np.repeat(dm["S_ITENS"].to_numpy(), pesos),	Compara variabilidade diária atual e planejada e percentual de saídas acima da jornada.
831	"H_TOTAL": np.repeat(dm["S_H_TOTAL"].to_numpy(), pesos), "KM": np.repeat(dm["S_KM"].to_numpy(), pesos)}	Compara variabilidade diária atual e planejada e percentual de saídas acima da jornada.

Linha	Código	Explicação
832	linhas = [("Diário", nome, "Entre dias de saída (CV da carga por saída)", cv(hist_dia[med]), cv(plano[med]))	Compara variabilidade diária atual e planejada e percentual de saídas acima da jornada.
833	for med, nome in MEDIDAS_IND.items()]	Compara variabilidade diária atual e planejada e percentual de saídas acima da jornada.
834	linhas.append(("Diário", "Horas trabalhadas", f"% das saídas acima de {JORNADA_MAX_H:g} h",	Compara variabilidade diária atual e planejada e percentual de saídas acima da jornada.
835	float((hist_dia["H_TOTAL"] > JORNADA_MAX_H).mean()), float((plano["H_TOTAL"] > JORNADA_MAX_H + 1e-9).mean()))	Compara variabilidade diária atual e planejada e percentual de saídas acima da jornada.
836	return linhas	Compara variabilidade diária atual e planejada e percentual de saídas acima da jornada.
837	``	Linha em branco usada para separar blocos.
838	``	Linha em branco usada para separar blocos.
839	# ---------- GRÁFICOS E EXPORTAÇÃO ----------	Gera gráficos PNG comparando atual e plano por mês e por fiscal para horas, quilômetros, itens e locais.
840	def graficos(comp: pd.DataFrame, anual: pd.DataFrame, hist_anual: pd.DataFrame, pasta: str):	Gera gráficos PNG comparando atual e plano por mês e por fiscal para horas, quilômetros, itens e locais.

Linha	Código	Explicação
841	`    try:`	Gera gráficos PNG comparando atual e plano por mês e por fiscal para horas, quilômetros, itens e locais.
842	`        import matplotlib`	Gera gráficos PNG comparando atual e plano por mês e por fiscal para horas, quilômetros, itens e locais.
843	`        matplotlib.use("Agg")`	Gera gráficos PNG comparando atual e plano por mês e por fiscal para horas, quilômetros, itens e locais.
844	`        import matplotlib.pyplot as plt`	Gera gráficos PNG comparando atual e plano por mês e por fiscal para horas, quilômetros, itens e locais.
845	`    except ImportError:`	Gera gráficos PNG comparando atual e plano por mês e por fiscal para horas, quilômetros, itens e locais.
846	`        return`	Gera gráficos PNG comparando atual e plano por mês e por fiscal para horas, quilômetros, itens e locais.
847	`    meses = ["jan", "fev", "mar", "abr", "mai", "jun", "jul", "ago", "set", "out", "nov", "dez"]`	Gera gráficos PNG comparando atual e plano por mês e por fiscal para horas, quilômetros, itens e locais.
848	`    titulos = {"H_TOTAL": "Horas trabalhadas", "KM": "Quilômetros", "ITENS": "Itens fiscalizados", "LOCAIS": "Locais fiscalizados"}`	Gera gráficos PNG comparando atual e plano por mês e por fiscal para horas, quilômetros, itens e locais.

Linha	Código	Explicação
849	`    fig, axs = plt.subplots(2, 2, figsize=(12, 8))`	Gera gráficos PNG comparando atual e plano por mês e por fiscal para horas, quilômetros, itens e locais.
850	`    for ax, med in zip(axs.ravel(), titulos):`	Gera gráficos PNG comparando atual e plano por mês e por fiscal para horas, quilômetros, itens e locais.
851	`        ax.bar(np.arange(12) - 0.2, comp[f"ATUAL_{med}"], 0.4, label="Atual", color="#B03A2E")`	Gera gráficos PNG comparando atual e plano por mês e por fiscal para horas, quilômetros, itens e locais.
852	`        ax.bar(np.arange(12) + 0.2, comp[f"PLANO_{med}"], 0.4, label="Plano (meta)", color="#2E6F9E")`	Gera gráficos PNG comparando atual e plano por mês e por fiscal para horas, quilômetros, itens e locais.
853	`        ax.set_xticks(range(12)); ax.set_xticklabels(meses); ax.set_title(f"{titulos[med]} da equipe por mês"); ax.legend(fontsize=8)`	Gera gráficos PNG comparando atual e plano por mês e por fiscal para horas, quilômetros, itens e locais.
854	`    fig.tight_layout(); fig.savefig(os.path.join(pasta, f"{NOME_SAIDA}_Mensal.png"), dpi=140); plt.close(fig)`	Gera gráficos PNG comparando atual e plano por mês e por fiscal para horas, quilômetros, itens e locais.
855	```	Linha em branco usada para separar blocos.
856	`    d = anual.merge(hist_anual.add_prefix("ATUAL_").rename(columns={"ATUAL_PESSOA": "PESSOA"}), on="PESSOA", how="left").fillna(0)`	Gera gráficos PNG comparando atual e plano por mês e por fiscal para horas, quilômetros, itens e locais.

Linha	Código	Explicação
857	`    d = d.sort_values("H_TOTAL")`	Gera gráficos PNG comparando atual e plano por mês e por fiscal para horas, quilômetros, itens e locais.
858	`    nomes = d["PESSOA"].str.title().apply(lambda s: f"{s.split()[0]} {s.split()[-1]}" if len(s.split()) > 1 else s)`	Gera gráficos PNG comparando atual e plano por mês e por fiscal para horas, quilômetros, itens e locais.
859	`    fig, axs = plt.subplots(2, 2, figsize=(12, 8))`	Gera gráficos PNG comparando atual e plano por mês e por fiscal para horas, quilômetros, itens e locais.
860	`    for ax, med in zip(axs.ravel(), titulos):`	Gera gráficos PNG comparando atual e plano por mês e por fiscal para horas, quilômetros, itens e locais.
861	`        y = np.arange(len(d))`	Gera gráficos PNG comparando atual e plano por mês e por fiscal para horas, quilômetros, itens e locais.
862	`        ax.barh(y + 0.2, d[f"ATUAL_{med}"], 0.4, label="Atual", color="#B03A2E")`	Gera gráficos PNG comparando atual e plano por mês e por fiscal para horas, quilômetros, itens e locais.
863	`        ax.barh(y - 0.2, d[med], 0.4, label="Plano (meta anual)", color="#2E6F9E")`	Gera gráficos PNG comparando atual e plano por mês e por fiscal para horas, quilômetros, itens e locais.
864	`        ax.set_yticks(y); ax.set_yticklabels(nomes, fontsize=8); ax.set_title(f"{titulos[med]} por fiscal no ano"); ax.legend(fontsize=7)`	Gera gráficos PNG comparando atual e plano por mês e por fiscal para horas, quilômetros, itens e locais.

Linha	Código	Explicação
865	`    fig.tight_layout(); fig.savefig(os.path.join(pasta, f"{NOME_SAIDA}_Fiscais.png"), dpi=140); plt.close(fig)`	Gera gráficos PNG comparando atual e plano por mês e por fiscal para horas, quilômetros, itens e locais.
866	``	Linha em branco usada para separar blocos.
867	``	Linha em branco usada para separar blocos.
868	`def exportar(abas: "OrderedDict"):`	Exporta cada aba em CSV e todas as abas em XLSX, registrando falhas individuais e aplicando formatação.
869	`    caminho_xlsx = os.path.join(PASTA, f"{NOME_SAIDA}.xlsx")`	Exporta cada aba em CSV e todas as abas em XLSX, registrando falhas individuais e aplicando formatação.
870	`    falhas = []`	Exporta cada aba em CSV e todas as abas em XLSX, registrando falhas individuais e aplicando formatação.
871	`    if EXPORTAR_CSV_POR_ABA:`	Exporta cada aba em CSV e todas as abas em XLSX, registrando falhas individuais e aplicando formatação.
872	`        pasta_csv = os.path.join(PASTA, f"{NOME_SAIDA}_csv")`	Exporta cada aba em CSV e todas as abas em XLSX, registrando falhas individuais e aplicando formatação.

Linha	Código	Explicação
873	`    os.makedirs(pasta_csv, exist_ok=True)`	Exporta cada aba em CSV e todas as abas em XLSX, registrando falhas individuais e aplicando formatação.
874	`    for nome, tab in abas.items():`	Exporta cada aba em CSV e todas as abas em XLSX, registrando falhas individuais e aplicando formatação.
875	`        try:`	Exporta cada aba em CSV e todas as abas em XLSX, registrando falhas individuais e aplicando formatação.
876	`            tab.to_csv(os.path.join(pasta_csv, f"{nome}.csv"), index=False, encoding="utf-8-sig")`	Exporta cada aba em CSV e todas as abas em XLSX, registrando falhas individuais e aplicando formatação.
877	`        except Exception as e:`	Exporta cada aba em CSV e todas as abas em XLSX, registrando falhas individuais e aplicando formatação.
878	`            falhas.append(f"csv/{nome}: {e}")`	Exporta cada aba em CSV e todas as abas em XLSX, registrando falhas individuais e aplicando formatação.
879	`    print(f"[OK] {NOME_SAIDA}_csv/ ({len(abas)} arquivos .csv)")`	Exporta cada aba em CSV e todas as abas em XLSX, registrando falhas individuais e aplicando formatação.

Linha	Código	Explicação
880	`    with pd.ExcelWriter(caminho_xlsx, engine="openpyxl") as w:`	Exporta cada aba em CSV e todas as abas em XLSX, registrando falhas individuais e aplicando formatação.
881	`        for nome, tab in abas.items():`	Exporta cada aba em CSV e todas as abas em XLSX, registrando falhas individuais e aplicando formatação.
882	`            aba = nome[:31]`	Exporta cada aba em CSV e todas as abas em XLSX, registrando falhas individuais e aplicando formatação.
883	`            try:`	Exporta cada aba em CSV e todas as abas em XLSX, registrando falhas individuais e aplicando formatação.
884	`                limpar_texto(tab).to_excel(w, sheet_name=aba, index=False)`	Exporta cada aba em CSV e todas as abas em XLSX, registrando falhas individuais e aplicando formatação.
885	`            except Exception as e:`	Exporta cada aba em CSV e todas as abas em XLSX, registrando falhas individuais e aplicando formatação.
886	`                falhas.append(f"xlsx/{nome}: {e}")`	Exporta cada aba em CSV e todas as abas em XLSX, registrando falhas individuais e aplicando formatação.

Linha	Código	Explicação
887	`            if aba in w.book.sheetnames:`	Exporta cada aba em CSV e todas as abas em XLSX, registrando falhas individuais e aplicando formatação.
888	`                del w.book[aba]`	Exporta cada aba em CSV e todas as abas em XLSX, registrando falhas individuais e aplicando formatação.
889	`        try:`	Exporta cada aba em CSV e todas as abas em XLSX, registrando falhas individuais e aplicando formatação.
890	`            formatar_planilhas(w)`	Exporta cada aba em CSV e todas as abas em XLSX, registrando falhas individuais e aplicando formatação.
891	`        except Exception as e:`	Exporta cada aba em CSV e todas as abas em XLSX, registrando falhas individuais e aplicando formatação.
892	`            print(f"AVISO: formatação não aplicada ({e})")`	Exporta cada aba em CSV e todas as abas em XLSX, registrando falhas individuais e aplicando formatação.
893	`    print(f"[OK] {NOME_SAIDA}.xlsx ({len(abas) - len(falhas)} abas)")`	Exporta cada aba em CSV e todas as abas em XLSX, registrando falhas individuais e aplicando formatação.

Linha	Código	Explicação
894	for f in falhas:	Exporta cada aba em CSV e todas as abas em XLSX, registrando falhas individuais e aplicando formatação.
895	print("[FALHOU]", f)	Exporta cada aba em CSV e todas as abas em XLSX, registrando falhas individuais e aplicando formatação.
896	return falhas	Exporta cada aba em CSV e todas as abas em XLSX, registrando falhas individuais e aplicando formatação.
897	``	Linha em branco usada para separar blocos.
898	``	Linha em branco usada para separar blocos.
899	def limpar_texto(df: pd.DataFrame) -> pd.DataFrame:	Remove caracteres de controle que poderiam impedir a gravação no Excel.
900	from openpyxl.cell.cell import ILLEGAL_CHARACTERS_RE	Remove caracteres de controle que poderiam impedir a gravação no Excel.
901	df = df.copy()	Remove caracteres de controle que poderiam impedir a gravação no Excel.
902	for c in df.columns:	Remove caracteres de controle que poderiam impedir a gravação no Excel.

Linha	Código	Explicação
903	`    if pd.api.types.is_object_dtype(df[c]) or pd.api.types.is_string_dtype(df[c]):`	Remove caracteres de controle que poderiam impedir a gravação no Excel.
904	`        df[c] = df[c].map(lambda x: ILLEGAL_CHARACTERS_RE.sub("", x) if isinstance(x, str) else x)`	Remove caracteres de controle que poderiam impedir a gravação no Excel.
905	`  return df`	Remove caracteres de controle que poderiam impedir a gravação no Excel.
906	``	Linha em branco usada para separar blocos.
907	``	Linha em branco usada para separar blocos.
908	`# ---------- PIPELINE ----------`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
909	`def main():`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.

Linha	Código	Explicação
910	`    try:`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
911	`        from google.colab import drive`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
912	`        drive.mount("/content/drive")`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
913	`    except ImportError:`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.

Linha	Código	Explicação
914	`    pass`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
915	``	Linha em branco usada para separar blocos.
916	`    print("== Lendo", os.path.basename(ARQ_BASE))`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
917	`    df, vis, fonte_ret = ler_base(ARQ_BASE)`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.

Linha	Código	Explicação
918	__reat = (df[df["FISCAL"] != df["FISCAL_ORIG"]]	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
919	_____.groupby(["FISCAL_ORIG", "FISCAL", "DIA", "ID_EQUIPE", "MUNICIPIO"])	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
920	_____.agg(ITENS=("CODITEM", "size"), LOCAIS=("COD_PROPR", "nunique")).reset_index()	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
921	_____.rename(columns={"FISCAL_ORIG": "FISCAL_REMOVIDO", "FISCAL": "FISCAL_QUE_ABSORVEU"}))	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.

Linha	Código	Explicação
922	reat["DIA"] = reat["DIA"].dt.date	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
923	removidos = sorted(set(df["FISCAL_ORIG"]) - set(df["FISCAL"]))	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
924	if removidos:	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
925	print(f"Removidos da planilha ({len(removidos)}): {', '.join(removidos)} | trabalho reatribuído: "	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.

Linha	Código	Explicação
926	f"{int(reat['ITENS'].sum())} itens em {int(reat['LOCAIS'].sum())} locais-dia (total da base inalterado: {len(df)} itens)")	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
927	ano = ANO_PLANO or int(df["DIA"].dt.year.mode()[0])	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
928	cal = montar_calendario(ano)	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
929	fiscais_hist = sorted(df["FISCAL"].unique()) # todos os fiscais da base ("como era")	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.

Linha	Código	Explicação
930	`    fiscais = resolver_fiscais(FISCAIS_PLANO, fiscais_hist) if FISCAIS_PLANO else list(fiscais_hist)`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
931	`    fiscais = [f for f in fiscais if f not in FISCAIS_EXCLUIR]`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
932	`    fora = [f for f in fiscais_hist if f not in fiscais]`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
933	`    print(f"{len(df)} itens | {len(vis)} visitas | {df['COD_PROPR'].nunique()} locais | {df['MUNICIPIO'].nunique()} municípios | "`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.

Linha	Código	Explicação
934	`        f"{len(fiscais)} fiscais no plano (base: {len(fiscais_hist)}) | ano {ano} | dias úteis {int(cal['DIA_UTIL'].sum())}")`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
935	`    if fora:`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
936	`        print("Fora do plano (a demanda deles é redistribuída):", ", ".join(fora))`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
937	``	Linha em branco usada para separar blocos.

Linha	Código	Explicação
938	`    tempos, tempo_faixas, tempo_resumo = calibrar_tempo(vis)`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
939	`    print("Tempo por visita (min) por faixa de itens:", {k: round(v, 1) for k, v in tempos.items()})`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
940	`    vis = aplicar_tempos(vis, tempos)`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
941	`    hist_dia = historico_dia(vis)`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.

Linha	Código	Explicação
942	`__globals().update(df=df, vis=vis, hist_dia=hist_dia)`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
943	``	Linha em branco usada para separar blocos.
944	`__ferias = ler_ferias(ARQ_FERIAS, fiscais, [])`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
945	`__disp_hist = montar_disponibilidade(cal, fiscais_hist, ferias)  # disponibilidade de todos (para o "Atual")`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.

Linha	Código	Explicação
946	`    disp = disp_hist[fiscais]                                    # disponibilidade de quem fará o plano`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
947	`    dias_f = disp.sum().reindex(fiscais).values`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
948	``	Linha em branco usada para separar blocos.
949	`    dm = montar_demanda(vis)`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.

Linha	Código	Explicação
950	__zonas = montar_zonas(dm)	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
951	__zonas_idx = zonas["ZONA_ID"].astype(int).values - 1	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
952	__print(f"Demanda: {int(dm['DEMANDA_VISITAS'].sum())} visitas em {int(dm['SAIDAS'].sum())} saídas")	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
953	``	Linha em branco usada para separar blocos.

Linha	Código	Explicação
954	`    print("\n== Otimização: divisão dos municípios entre os fiscais")`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
955	`    x, info = resolver(fiscais, dias_f, dm, zonas_idx, "fiscais")`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
956	`    print(f"Método: {info['metodo']} | desvios máximos: { {k: round(v, 3) for k, v in info['desvio_max'].items()} }")`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
957	`    est = atribuir_locais(x, fiscais, dm, vis, df)`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.

Linha	Código	Explicação
958	`    x, ajustes = corrigir_saidas_sem_locais(x, fiscais, dm, est)`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
959	`    longo = plano_longo(x, fiscais, dm, est, vis)`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
960	`    anual = plano_anual(longo, fiscais, dias_f)`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
961	``	Linha em branco usada para separar blocos.

Linha	Código	Explicação
962	__mes_t = metas_periodicas(anual, disp, cal, "MES", True)	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
963	__sem_t = metas_periodicas(anual, disp, cal, "SEMANA", False)	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
964	``	Linha em branco usada para separar blocos.
965	__gm = grade_periodos(hist_dia, fiscais_hist, cal, "MES") # "Atual": todos os fiscais da base	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.

Linha	Código	Explicação
966	`    gs = grade_periodos(hist_dia, fiscais_hist, cal, "SEMANA")`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
967	`    ref = gm[["PESSOA", "MES"] + MEDIDAS].rename(columns={m: f"ATUAL_{m}" for m in MEDIDAS})`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
968	`    mes_t = mes_t.merge(ref, on=["PESSOA", "MES"], how="left")`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
969	`    ind_pessoas = indicadores_pessoas(hist_dia, fiscais_hist, disp_hist, cal)`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.

Linha	Código	Explicação
970	``	Linha em branco usada para separar blocos.
971	__atual = indicadores_grupo(gm, gs, disp_hist, cal, fiscais_hist)	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
972	__plano = indicadores_grupo(mes_t[["PESSOA", "MES"] + MEDIDAS], sem_t[["PESSOA", "SEMANA"] + MEDIDAS], disp, cal, fiscais)	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
973	__indic = [a + (p[-1],) for a, p in zip(atual, plano)] + indicador_diario(hist_dia, dm)	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.

Linha	Código	Explicação
974	`    indic = pd.DataFrame(indic, columns=["NIVEL", "MEDIDA", "INDICADOR (menor = mais equilibrado)", "ATUAL", "PLANO"])`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
975	`    indic["REDUCAO_%"] = (indic["ATUAL"] - indic["PLANO"]) / indic["ATUAL"].replace(0, np.nan)`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
976	``	Linha em branco usada para separar blocos.
977	`    # equipe: atual x plano por mês`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.

Linha	Código	Explicação
978	__comp = pd.DataFrame({"MES": range(1, 13)}).set_index("MES")	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
979	__plano_mes = mes_t.groupby("MES")[MEDIDAS].sum()	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
980	__atual_mes = gm.groupby("MES")[MEDIDAS].sum()	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
981	__for med in MEDIDAS:	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.

Linha	Código	Explicação
982	`    comp[f"ATUAL_{med}"] = atual_mes[med].reindex(comp.index).fillna(0)`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
983	`    comp[f"PLANO_{med}"] = plano_mes[med].reindex(comp.index).fillna(0)`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
984	`  comp = comp.reset_index()`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
985	`  uteis = cal.groupby("MES")["DIA_UTIL"].sum()`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.

Linha	Código	Explicação
986	`    cap = pd.DataFrame({"MES": range(1, 13)})`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
987	`    cap["DIAS_UTEIS"] = cap["MES"].map(uteis).astype(int)`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
988	`    cap["SAIDAS_PLANO"] = cap["MES"].map(plano_mes["SAIDAS"])`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
989	`    cap["EQUIPES_POR_DIA_UTIL_PLANO"] = cap["SAIDAS_PLANO"] / cap["DIAS_UTEIS"]`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.

Linha	Código	Explicação
990	`    cap["FISCAIS_DISPONIVEIS_POR_DIA"] = cap["MES"].map(disp.groupby(cal.set_index("DATA")["MES"].reindex(disp.index)).sum().sum(axis=1)) / cap["DIAS_UTEIS"]`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
991	`    cap["FOLGA_FISCAIS_POR_DIA"] = cap["FISCAIS_DISPONIVEIS_POR_DIA"] - cap["EQUIPES_POR_DIA_UTIL_PLANO"]`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
992	``	Linha em branco usada para separar blocos.
993	`    # responsáveis por município`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.

Linha	Código	Explicação
994	`    mat = longo.pivot_table(index="MUNICIPIO", columns="PESSOA", values="SAIDAS", aggfunc="sum", fill_value=0).reindex(dm["MUNICIPIO"]).fillna(0).astype(int)`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
995	`    resp = pd.DataFrame({"MUNICIPIO": mat.index, "SAIDAS_TOTAL": mat.sum(axis=1).values, "RESPONSAVEL_PRINCIPAL": mat.idxmax(axis=1).values,`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
996	`                "PCT_PRINCIPAL": (mat.max(axis=1) / mat.sum(axis=1)).values, "N_RESPONSAVEIS": (mat > 0).sum(axis=1).values})`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
997	`    resp["DEMAIS_RESPONSAVEIS"] = [", ".join(f"{p.title()} ({int(mat.loc[m, p])})" for p in mat.columns`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.

Linha	Código	Explicação
998	if mat.loc[m, p] > 0 and p != resp.loc[i, "RESPONSAVEL_PRINCIPAL"]) for i, m in enumerate(mat.index)]	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
999	resp = resp.merge(zonas[["MUNICIPIO", "ZONA"]], on="MUNICIPIO", how="left")	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1000	resp = resp.merge(dm[["MUNICIPIO", "ESTABELECIMENTOS", "VISITAS", "ITENS"]], on="MUNICIPIO", how="left")	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1001	mat_x = mat.reset_index(); mat_x["TOTAL"] = mat.sum(axis=1).values	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.

Linha	Código	Explicação
1002	``	Linha em branco usada para separar blocos.
1003	__# itens por espécie x fiscal (plano)	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1004	__esp = df.merge(est[["COD_PROPR", "FISCAL_PLANO"]], on="COD_PROPR", how="left")	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1005	__itens_esp = esp.pivot_table(index="ESPECIE", columns="FISCAL_PLANO", values="CODITEM", aggfunc="size", fill_value=0)	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.

Linha	Código	Explicação
1006	__itens_esp["TOTAL"] = itens_esp.sum(axis=1)	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1007	__itens_esp = itens_esp.sort_values("TOTAL", ascending=False).reset_index()	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1008	``	Linha em branco usada para separar blocos.
1009	__locais_x = est[["COD_PROPR", "NOMEFANTASIA", "ENDERECO", "MUNICIPIO", "BAIRRO", "LATITUDE", "LONGITUDE", "PRECISAO_GEO",	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.

Linha	Código	Explicação
1010	"VISITAS", "ITENS", "FISCAL_HISTORICO", "FISCAL_PLANO", "MUDOU_DE_FISCAL"]].rename(columns={"VISITAS": "VISITAS_NO_PERIODO", "ITENS": "ITENS_NO_PERIODO"})	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1011	dem_x = dm[["MUNICIPIO", "ESTABELECIMENTOS", "VISITAS", "ITENS", "DEMANDA_VISITAS", "VISITAS_POR_SAIDA_HIST", "VISITAS_POR_SAIDA", "SAIDAS",	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1012	"R_H", "R_KM", "FONTE_VIAGEM", "LEG_MIN", "LEG_KM", "TF_MEDIO_H", "S_LOCAIS", "S_ITENS", "S_H_FISC", "S_H_DESLOC", "S_H_TOTAL", "S_KM",	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1013	"H_TOTAL_PLANO", "KM_PLANO", "SAIDA_EXCEDE_JORNADA", "PCT_GEO_BAIXA_PRECISAO", "REVISAR_DESLOCAMENTO"]].rename(columns={	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.

Linha	Código	Explicação
1014	"R_H": "VIAGEM_REDONDA_H", "R_KM": "VIAGEM_REDONDA_KM", "LEG_MIN": "MIN_ENTRE_VISITAS_NO_MUN", "LEG_KM": "KM_ENTRE_VISITAS_NO_MUN",	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1015	"S_LOCAIS": "LOCAIS_POR_SAIDA", "S_ITENS": "ITENS_POR_SAIDA", "S_H_FISC": "H_FISCALIZACAO_POR_SAIDA", "S_H_DESLOC": "H_DESLOC_POR_SAIDA",	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1016	"S_H_TOTAL": "H_TOTAL_POR_SAIDA", "S_KM": "KM_POR_SAIDA"})	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1017	``	Linha em branco usada para separar blocos.

Linha	Código	Explicação
1018	`    disp_mes = disp.groupby(cal.set_index("DATA")["MES"].reindex(disp.index)).sum().T`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1019	`    disp_mes.columns = [f"MES_{c:02d}" for c in disp_mes.columns]`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1020	`    disp_mes["TOTAL_ANO"] = disp_mes.sum(axis=1)`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1021	`    disp_mes = disp_mes.reset_index().rename(columns={"index": "FISCAL"})`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.

Linha	Código	Explicação
1022	``	Linha em branco usada para separar blocos.
1023	__h_atual = hist_dia.groupby("PESSOA")[["LOCAIS", "ITENS", "H_TOTAL", "KM"]].sum().reindex(fiscais_hist).fillna(0)	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1024	__gini_a, gini_p = gini(h_atual["H_TOTAL"]), gini(anual["H_TOTAL"])	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1025	__parametros = pd.DataFrame([	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.

Linha	Código	Explicação
1026	("Ano do plano", ano, ""), ("Fiscais no plano", len(fiscais), ", ".join(fiscais)),	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1027	("Fiscais removidos da planilha", len(removidos), (", ".join(removidos) + " (trabalho reatribuído; total de fiscalizações inalterado)") if removidos else "nenhum"),	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1028	("Fiscais da base fora do plano", len(fora), (", ".join(fora) + " (demanda redistribuída)") if fora else "nenhum"),	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1029	("Retorno à central", fonte_ret, f"Central: {SEDE_ENDERECO}"),	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.

Linha	Código	Explicação
1030	("Coordenada da central", f"{SEDE[0]}, {SEDE[1]}", "Usada só para estimar retorno/viagem quando a base não tem"),	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1031	("Viagem à central: circuidade / velocidade medidas na base", f"{dm.attrs.get('circuidade_base', np.nan):.2f} / {dm.attrs.get('velocidade_base', np.nan):.0f} km/h",	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1032	"Usadas só nos municípios sem trecho real na base"),	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1033	("Compartilhamento da viagem (dias com mais de um município)", f"{dm.attrs.get('compartilhamento', 1.0):.3f}", "Multiplica a viagem à central de cada saída"),	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.

Linha	Código	Explicação
1034	("Tempo por visita (min) por faixa de itens", str({k: round(v, 1) for k, v in tempos.items()}), "Calibrado no período (aba Tempo_Fiscalizacao"),	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1035	("JORNADA_MAX_H", JORNADA_MAX_H, "Limite de horas de uma saída"), ("FATOR_DEMANDA", FATOR_DEMANDA, "1.0 = mesma demanda do período"),	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1036	("Pesos do equilíbrio", str(PESOS), "Locais, itens, horas trabalhadas e km"),	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1037	("N_ZONAS", N_ZONAS, ""), ("Municípios por fiscal (mín / máx)", f"{MIN_MUN_POR_FISCAL} / {MAX_MUN_POR_FISCAL}", ""),	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.

Linha	Código	Explicação
1038	("Responsáveis por município (máx)", MAX_FISCAIS_POR_MUN, ""), ("MIN_SAIDAS_BLOCO", MIN_SAIDAS_BLOCO, ""),	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1039	("TOLERANCIA_META", TOLERANCIA_META, "Faixa mínimo/máximo das metas"),	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1040	("Períodos de férias considerados", len(ferias), os.path.basename(ARQ_FERIAS)),	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1041	("Método de otimização", info["metodo"], info["mensagem"][:120])], columns=["Parâmetro", "Valor", "Observação"])	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.

Linha	Código	Explicação
1042	``	Linha em branco usada para separar blocos.
1043	resumo = pd.DataFrame([	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1044	("Demanda atendida", f"{int(anual['LOCAIS'].sum())} locais/visitas e {int(anual['ITENS'].sum())} itens = 100% do período, por {len(fiscais)} fiscais"),	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1045	("Fiscais fora do plano", (", ".join(fora) + ": a demanda deles foi redistribuída") if fora else "nenhum"),	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.

Linha	Código	Explicação
1046	("Locais (estabelecimentos) divididos", f"{est['COD_PROPR'].nunique()} (mudam de fiscal: {int(est['MUDOU_DE_FISCAL'].sum())})"),	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1047	("Saídas (dias de equipe) no plano", f"{int(anual['SAIDAS'].sum())} (período: {len(hist_dia)} fiscal-dias)"),	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1048	("Horas trabalhadas: plano x período", f"{anual['H_TOTAL'].sum():.0f} h x {hist_dia['H_TOTAL'].sum():.0f} h"),	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1049	("Quilômetros: plano x período", f"{anual['KM'].sum():.0f} km x {hist_dia['KM'].sum():.0f} km"),	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.

Linha	Código	Explicação
1050	("Horas por fiscal no plano (mín / média / máx)", f"{anual['H_TOTAL'].min():.0f} / {anual['H_TOTAL'].mean():.0f} / {anual['H_TOTAL'].max():.0f} h"),	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1051	(f"Horas por fiscal no período, {len(fiscais_hist)} fiscais (mín / média / máx)", f"{h_atual['H_TOTAL'].min():.0f} / {h_atual['H_TOTAL'].mean():.0f} / {h_atual['H_TOTAL'].max():.0f} h"),	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1052	(f"Itens por fiscal no plano, {len(fiscais)} fiscais (mín / máx)", f"{anual['ITENS'].min()} / {anual['ITENS'].max()} (período: {int(h_atual['ITENS'].min())} / {int(h_atual['ITENS'].max())})"),	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1053	("Locais por fiscal no plano (mín / máx)", f"{anual['LOCAIS'].min()} / {anual['LOCAIS'].max()} (período: {int(h_atual['LOCAIS'].min())} / {int(h_atual['LOCAIS'].max())})"),	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.

Linha	Código	Explicação
1054	(f"Gini das horas entre fiscais: período ({len(fiscais_hist)}) x plano ({len(fiscais)})", f"{gini_a:.3f} x {gini_p:.3f}"),	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1055	("Municípios com mais de um responsável", f"{int((resp['N_RESPONSAVEIS'] > 1).sum())} de {len(resp)}"),	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1056	("Dias de saída por fiscal (média) / % dos dias úteis", f"{anual['SAIDAS'].mean():.0f} dias / {anual['PCT_DIAS_COM_SAIDA'].mean():.0%}"),	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1057	("Férias informadas", "SIM" if len(ferias) else "NÃO: modelo criado para preenchimento; plano sem férias"),	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.

Linha	Código	Explicação
1058	("Municípios para revisar o deslocamento", ", ".join(dm.loc[dm["REVISAR_DESLOCAMENTO"], "MUNICIPIO"]) or "nenhum"),	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1059	("Municípios com saída acima da jornada", ", ".join(dm.loc[dm["SAIDA_EXCEDE_JORNADA"], "MUNICIPIO"]) or "nenhum"),	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1060	("Observação", "Os valores do PLANO são METAS proporcionais aos dias disponíveis; o realizado depende da programação diária.")],	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1061	columns=["Item", "Resultado"])	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.

Linha	Código	Explicação
1062	``	Linha em branco usada para separar blocos.
1063	__abas = OrderedDict()	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1064	__abas["Resumo"] = resumo	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1065	__abas["Parametros"] = parametros	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.

Linha	Código	Explicação
1066	`    abas["Indicadores_Equilibrio"] = arredondar(indic)`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1067	`    abas["Plano_Fiscal_Anual"] = arredondar(anual.rename(columns={"PESSOA": "FISCAL"}))`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1068	`    abas["Responsavel_por_Municipio"] = arredondar(resp)`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1069	`    abas["Matriz_Saidas_Mun_x_Fiscal"] = mat_x`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.

Linha	Código	Explicação
1070	`__abas["Plano_Municipio_Fiscal"] = arredondar(longo.rename(columns={"PESSOA": "FISCAL"}))`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1071	`__abas["Locais_por_Fiscal"] = arredondar(locais_x)`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1072	`__abas["Itens_Especie_x_Fiscal"] = itens_esp`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1073	`__abas["Metas_Fiscal_Mensal"] = arredondar(mes_t.rename(columns={"PESSOA": "FISCAL"}))`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.

Linha	Código	Explicação
1074	__abas["Metas_Fiscal_Semanal"] = arredondar(sem_t.rename(columns={"PESSOA": "FISCAL"}))	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1075	__abas["Atual_Mensal_x_Plano"] = arredondar(comp)	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1076	__abas["Capacidade_Equipes"] = arredondar(cap)	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1077	__abas["Equilibrio_Atual_Fiscais"] = arredondar(ind_pessoas)	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.

Linha	Código	Explicação
1078	__abas["Demanda_Municipio"] = arredondar(dem_x)	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1079	__abas["Zonas"] = arredondar(zonas.drop(columns=["ZONA_ID"]))	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1080	__abas["Tempo_Fiscalizacao"] = arredondar(tempo_resumo.assign(Valor=tempo_resumo["Valor"].astype(float)))	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1081	__abas["Tempo_por_Itens"] = arredondar(tempo_faixas)	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.

Linha	Código	Explicação
1082	`    abas["Disponibilidade"] = disp_mes`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1083	`    abas["Ferias_Consideradas"] = ferias.assign(INICIO=ferias["INICIO"].dt.date, FIM=ferias["FIM"].dt.date) if len(ferias) else ferias`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1084	`    abas["Calendario"] = cal.assign(DATA=cal["DATA"].dt.date)`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1085	`    if len(reat):`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.

Linha	Código	Explicação
1086	`    abas["Reatribuicao_Removidos"] = reat`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1087	```	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1088	`  print("\n== Exportação")`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1089	`  falhas = exportar(abas)`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.

Linha	Código	Explicação
1090	__try:	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1091	____graficos(comp, anual, h_atual.reset_index().rename(columns={"PESSOA": "PESSOA", "FISCAL": "PESSOA"}), PASTA)	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1092	__except Exception as e:	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1093	____print(f"AVISO: gráficos não gerados ({type(e).__name__}: {e})")	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.

Linha	Código	Explicação
1094	``	Linha em branco usada para separar blocos.
1095	__pd.set_option("display.width", 220)	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1096	__print("\n=== RESUMO ===")	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1097	__print(resumo.to_string(index=False))	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.

Linha	Código	Explicação
1098	`    print("\n=== Plano anual por fiscal ===")`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1099	`    print(anual[["PESSOA", "SAIDAS", "LOCAIS", "ITENS", "H_TOTAL", "KM", "MUNICIPIOS"]].round(1).to_string(index=False))`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1100	`    print("\nConcluído." if not falhas else f"\nConcluído com {len(falhas)} aviso(s) de exportação.")`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1101	`    globals().update(abas=abas, dm=dm, zonas=zonas, est=est, x=x)`	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.

Linha	Código	Explicação
1102	``	Linha em branco usada para separar blocos.
1103	``	Linha em branco usada para separar blocos.
1104	if __name__ == "__main__":	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.
1105	__main()	Executa o pipeline principal: lê a base, reatribui removidos, define ano e fiscais, calibra tempos, monta demanda e zonas, resolve o plano, gera metas/indicadores/tabelas, exporta abas e gráficos e imprime o resumo final.

12. Conclusão

A programação constitui uma etapa de planejamento e apoio à decisão. Ela transforma dados históricos tratados em uma proposta operacional mensurável, permitindo comparar o cenário atual com metas de distribuição mais equilibradas. A qualidade da extração, do tratamento e da limpeza dos dados é diretamente determinante para a confiabilidade da otimização e dos indicadores apresentados no dashboard.